Linnaeus University Dissertations
No 639/2026

Rakhshanda Jabeen

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

LINNAEUS UNIVERSITY PRESS

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

Linnaeus University Dissertations

No 639/2026

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

Rakhshanda Jabeen

LINNAEUS UNIVERSITY PRESS

The doctoral dissertation seminar will be held at 9:15 am on 4 December 2025 in room D113, at Linnaeus University, Växjö. The opponent will be Docent Sebastian Herold, Karlstad University.

Representing Architectural Evidence in Software Systems: A Graph Learning Approach
Doctoral Dissertation, Department of Computer Science and Media Technology, Linnaeus University, Växjö, 2026

ISBN: 978-91-8082-518-4 (print), 978-91-8082-519-1 (pdf)
© Rakhshanda Jabeen
DOI: https://doi.org/10.15626/LUD.639.2026
Published by: Linnaeus University Press, 351 95 Växjö
Printed by: Multiply, Malmö, 2026

Abstract

Jabeen, Rakhshanda (2026). *Representing Architectural Evidence in Software Systems: A Graph Learning Approach,* Linnaeus University Dissertations No 639/2026, ISBN: 978-91-8082-518-4 (print), 978-91-8082-519-1 (pdf).

Software architecture documentation is often missing, incomplete, or outdated, making it unclear which source files implement which architectural modules. Software architecture recovery infers a modular decomposition from the implementation, while code-to-architecture mapping assigns files to known modules using a partial mapping. Both tasks can use dependencies, folder structure, and source code, but existing methods often rely on individual sources or predefined combinations. It remains unclear what each source contributes and when propagating information between connected files helps rather than mixes evidence from different modules.

The research uses open-source Java, C, and C++ systems with published reference decompositions. Recovery studies examine learning objectives, graph construction, and information sources, while mapping studies investigate representations, graph-based propagation, and iterative self-training.

For architecture recovery, learned representations achieve agreement with reference architectures comparable to or better than the evaluated baselines. In the evaluated configurations, reconstruction generally outperforms contrastive learning, while random walks and diffusion capture structural information beyond direct dependencies. However, combining structural, lexical, and folder information can improve agreement with reference architectures while producing modules with weaker dependency cohesion than those obtained from dependency-based random-walk embeddings.

For code-to-architecture mapping, neural classifiers outperform a Naive Bayes baseline. A graph attention network outperforms a graph-free classifier on most systems, although gains vary by system and configuration. Folder information produces the largest and most consistent improvements, but higher accuracy can coincide with misclassifications of

files that a simple directory rule assigns correctly. Representations that encode dependencies generally outperform lexical ones, and dependency-guided fine-tuning improves contextual representations.

However, strong graph-model performance does not by itself demonstrate a benefit from additional message passing. Controlled experiments show that propagating dependency information can reduce performance for some predictors compared with matched models without cross-file messages. Changes to graph construction, propagation methods, or learning settings can reduce these losses or lead to improvements.

Together, these findings show that the usefulness of architectural information depends not only on what is available, but also on how it is represented, combined, and used.

Keywords: Software Architecture, Software Architecture Recovery, Representation Learning, Graph Neural Networks, Graph Autoencoders

وَفَوْقَ كُلِّ ذِي عِلْمٍ عَلِيمٌ ﴿٧٦﴾

*"And above every possessor of knowledge is **one** more knowing."*

Quran, Surah Yusuf (12:76)

In memory of my mother, Safia Kausar.

Acknowledgments

First and foremost, I thank Allah for giving me the strength, patience, and health to begin this work and bring it to completion.

These five years have taught me patience and perseverance, and I thank my main supervisor, Morgan Ericsson, who has been a great mentor to me throughout this time. I have learned a great deal from him, and when I look back, it is the small details and teaching moments that stand out. He has helped me more than I can express here, and I am deeply grateful.

I also thank my co-supervisors, Jonas Nordqvist and Anna Wingkvist, for their guidance and feedback throughout this work. Welf Löwe, thank you for your careful review and constructive feedback, which helped improve this thesis.

At Electrolux Professional, I thank my managers, Dorothee Millon and Massimo Gerolami, for their support and for understanding the time and commitment required by my PhD studies. I also thank my colleagues at Linnaeus University and Electrolux Professional for their support throughout these years.

I am deeply grateful to my father, Imtiaz Ahmad, for his love, prayers, and for always believing in me. To my brothers, Yasir and Nasir, and my sister, Farkhanda, thank you for always being there for me, especially when things were difficult. Finally, I thank my friends for their encouragement and for the good times we have shared over these years.

Rakhshanda Jabeen
4th December, 2026
Växjö, Sweden

Table of Contents

Acknowledgments **xiii**

Table of Contents **xv**

List of Figures **xvii**

List of Tables **xix**

List of Publications **xxiii**

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Aim and Scope . . . 4
- 1.4 Thesis Contributions . . . 4
- 1.5 Thesis Outline . . . 5

2 Background **7**
- 2.1 Prior Approaches . . . 7
- 2.2 Representing Artifact Content . . . 11
- 2.3 Graph Representation Learning . . . 12
- 2.4 Evaluation and Ground Truth . . . 16

3 Problem Formulation and Research Objectives **19**
- 3.1 Problem Formulation . . . 19
- 3.2 Research Objectives . . . 20

4 Overview of the Studies **23**
- 4.1 Relationship Between Objectives and Studies . . . 23
- 4.2 Overview of the Included Studies . . . 23

5 Research Method **27**
- 5.1 Research Design . . . 27
- 5.2 Graph and Signal Representation . . . 30
- 5.3 Learning Settings . . . 32

5.4 Benchmark Systems 35
5.5 Evaluation Protocol 36
5.6 Cross-Study Evaluation Design 37

6 Research Findings and Synthesis **41**
6.1 Results of the Cross-Study Evaluations 41
6.2 Synthesis of Research Findings 46

7 Discussion **53**
7.1 Interpretation of the Findings 53
7.2 Limitations 56

8 Conclusion and Future Work **61**
8.1 Conclusion 61
8.2 Future Work 63

Bibliography **65**

Appended Publications

Graph Representation Learning for Software Architecture Recovery **71**

GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery **79**

When Does the Dependency Graph Help Code-to-Architecture Mapping? **103**

Graph Embeddings for Software Architecture Recovery **121**

What Carries Architectural Signal? A Code-to-Architecture Mapping Study **141**

List of Figures

4.1 A high-level overview of objectives mapped to the papers. 24

5.1 Design space studied in the thesis. 29
5.2 Node-set configurations used across the thesis. 31

6.1 Overview of recovery results across studies. 42
6.2 Overview of C2A mapping results across studies. 44

7.1 Correlation between architecture recovery metrics and system properties. 55

List of Tables

2.1 Taxonomy of representative prior approaches. 8

5.1 Graph construction and signal representation across thesis. 33
5.2 Encoder configurations and training settings across studies. . . . 34
5.3 Properties and versions of the benchmark systems. 35
5.4 Properties of the benchmark systems. 39

6.1 Recovered partitions compared with the ground truth in TMQ_{norm}. 44
6.2 Typed versus merged GAT across file representations. 45

List of Abbreviations

Tasks, Data and Background

AST	Abstract Syntax Tree
C2A	Code-to-architecture
MQ	Modularization quality
SAR	Software architecture recovery
SDG	Software dependency graph

Models and Graph Processing

GAE	Graph autoencoder
GAT	Graph attention network
GAT-DC	Graph attention network with graph diffusion convolution
GAT-E	Graph attention network with edge attributes
GCN	Graph convolutional network
GDC	Graph diffusion convolution
GNN	Graph neural network
HGAT	Heterogeneous graph attention network
HGAT-TD	Heterogeneous graph attention network with typed dependencies
HGT	Heterogeneous Graph Transformer
LPE	Laplacian positional encoding
M2V	MetaPath2Vec
MLP	Multilayer perceptron
N2V	Node2Vec
PPR	Personalized PageRank
VGAE	Variational graph autoencoder

Input Representations

BoW	Bag-of-words
UXC	UniXcoder
W2V	Word2Vec

Training Objectives

BCE	Binary cross-entropy
CCE	Categorical cross-entropy
DGI	Deep Graph Infomax
KL	Kullback–Leibler

Evaluation Metrics

A2A	Architecture-to-architecture
ARI	Adjusted Rand index
$\mathbf{C2C_{cvg}}$	Cluster-to-cluster coverage
MoJoFM	Move and Join effectiveness measure
$\mathbf{TMQ_{norm}}$	Normalized turbo modularization quality

List of Publications

This dissertation is based on the following publications. I contributed to all stages of the work as lead author.

I. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Representation Learning for Software Architecture Recovery". In: *ESANN*. 2026. (page 71).

II. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery". In: *Proceedings of the ACM on Software Engineering* 3.FSE (July 2026). DOI: 10.1145/3808187. (page 79).

III. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. *When Does the Dependency Graph Help Code-to-Architecture Mapping?* 2026. arXiv: 2610.19769 [cs.SE]. URL: https://arxiv.org/abs/2610.19769. *Under review.* (page 103).

IV. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Embeddings for Software Architecture Recovery". In: *Nordic Machine Intelligence* (2026). DOI: 10.5617/nmi.13153. URL: https://journals.uio.no/NMI/article/view/13153. (page 121).

V. Rakhshanda Jabeen, Morgan Ericsson, Yuyao Duan, and Anna Wingkvist. *What Carries Architectural Signal? A Code-to-Architecture Mapping Study.* 2026. arXiv: 2610.41628 [cs.SE]. URL: https://arxiv.org/abs/2610.41628. *Under review.* (page 141).

Chapter 1

Introduction

Contents

1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Aim and Scope 4
1.4 Thesis Contributions 4
1.5 Thesis Outline 5

Software architecture provides an abstraction of a software system in terms of its principal elements, their responsibilities, and the relations among them [10]. This abstraction supports reasoning about the system without requiring developers to inspect every implementation detail. However, the connection between the intended architecture and the source code is often incomplete or outdated [1, 13]. Recovering this connection relies on evidence from the implementation, which is distributed across multiple artifacts and their relations [8, 55]. Prior work either uses such evidence in isolation or combines multiple sources to support architectural analysis [87, 12, 8]. This thesis investigates how graph-based representation learning can use static implementation artifacts to support architectural analysis.

1.1 Motivation

Software architecture is often reflected in the implemented system as a modular decomposition: a partition of files into modules that group related functionality [68]. Such decompositions matter for program understanding, maintenance, and refactoring [10]. As a system evolves, developers introduce dependencies that were not anticipated in the original design, refactor code across module boundaries, and add functionality that fits no existing module. Documentation is often not maintained in step with these changes. The resulting gap between the intended design and the implementation is described using two related terms: *architectural drift*, where the implementation grows away from the intended design without directly violating it, and *architectural erosion*, where the implementation directly violates the intended design [13].

Architectural drift and erosion can contribute to *technical debt* by making later maintenance and change more costly [45]. This concern may become more relevant as large language models (LLMs) are increasingly used to assist software development. A multivocal literature review reports that LLM-assisted development can amplify existing forms of technical debt, including design debt [16]. An empirical study of AI-assisted development also reports short-term gains in development velocity alongside persistent increases in code complexity [28]. These findings provide additional motivation to assess architectural consistency, since changes that satisfy individual tasks may still conflict with the intended organization of responsibilities and dependencies across the system.

Two related activities address the gap between the intended architecture and its implementation. Software architecture recovery (SAR) is used when no sufficiently reliable architectural decomposition is available. It infers such a decomposition by grouping source-code files into architectural modules using evidence from the implementation. Performing recovery manually requires substantial knowledge of both the system and its domain, and becomes difficult as the number of files grows [20].

Code-to-architecture (C2A) mapping is used when the architectural modules are known, but the assignments of source-code files to those modules are absent, incomplete, or outdated. It assigns files to these predefined modules [81, 9]. The resulting mapping supports techniques such as reflexion modeling, which compare implemented dependencies with intended architectural relations [57]. Maintaining the mapping manually is costly because file assignments must be reconsidered as the implementation and architectural responsibilities evolve [1].

Although the two activities differ in the architectural information available at the outset, both use implementation evidence to assign files to architectural modules. Dependencies, project organization, and source-code content offer potentially complementary evidence, motivating an investigation into how these signals can support recovery and mapping.

1.2 Problem Statement

Both tasks assign source-code files to architectural modules, and a single implementation property does not determine a file's architectural role. Instead, evidence of module membership is distributed across several signals. Static dependencies describe how files call, import, inherit from, or otherwise use one another. Files that implement related responsibilities may be directly connected or occupy nearby regions of the dependency graph, making dependency structure a potential source of architectural information [8]. Dependency types can also describe different forms of interaction and may not contribute equally to architectural similarity [78].

Project organization, represented here by the folder hierarchy, provides a second signal. Folder hierarchies record how developers organize implementation artifacts and are commonly consulted when producing reference decompo-

sitions [20]. However, folders are implementation-level containers rather than architectural modules. A module can span several folders, and a folder can contain files belonging to several architectural concerns. Folder location, therefore, carries useful but imperfect evidence.

Source-code content provides a third signal. File names, identifiers, and comments encode domain-specific knowledge about the concepts and responsibilities a file implements [87]. Files from the same module may therefore share terminology, and lexical content has been used to group files by semantic similarity in prior recovery work [22, 12]. However, vocabulary can also recur across modules, particularly in shared abstractions and utility code. Lexical similarity is therefore informative, but may be insufficient on its own. These signals may provide complementary evidence, but their combination is not straightforward. A modular decomposition is expected to have high *cohesion* within modules and low *coupling* between them [54], and each signal is only an indirect indicator of these properties. A dependency can connect files within the same module or cross a module boundary. A folder can approximate a module or reflect only organizational conventions. Similar identifiers can indicate shared responsibility or merely the use of a common domain vocabulary. The strength of each signal can therefore vary across systems and modules.

Prior approaches, reviewed in Section 2.1, include dependency-based approaches [54, 80, 51, 8], lexical approaches [12, 22], and lexical models that encode dependencies as tokens [64]. Approaches that rely on a single source can leave potentially complementary evidence unused. To incorporate such evidence, several approaches combine multiple signals through predefined similarity functions or weighting schemes [87, 67, 53].

Graph neural networks (GNNs) offer a way to learn from these signals jointly by mapping the dependency graph, along with the attributes attached to its nodes and edges, into a common vector space. This does not by itself determine what is learned. The result depends on how files are represented, how the graph is constructed, how information is shared between connected files, and what objective drives learning. When several of these change at once, a difference in performance cannot be attributed to any one signal. Two further questions remain open: whether an added signal contributes beyond what the representation already encodes, and whether sharing information between connected files helps or instead mixes evidence from different modules when their relations cross architectural boundaries [89].

The problem addressed by this research is therefore not only how to combine implementation signals. It is to examine what architectural information each signal carries, how that information should be represented, how the signals interact when combined, and under which structural conditions propagation along the graph is beneficial. Answering this requires varying input representation, graph construction, relation modeling, and message passing separately, and examining how they interact.

1.3 Research Aim and Scope

This thesis investigates how graph-based representation learning can support software architecture recovery and code-to-architecture mapping. It examines the architectural information available in static implementation signals and how their usefulness changes with the input representation, graph structure, and message-passing mechanism.

This thesis aims to determine how graph-based representation learning can be designed for recovery and mapping, what architectural information the underlying implementation signals carry, and how the contribution of file representations changes when folder organization and member-level detail are added to the graph and dependency neighbors are made available through message passing. Both tasks are studied under a common perspective. Files and other implementation artifacts are represented as nodes, and their relations as edges; artifact-level information is provided as node attributes, and a graph representation-learning encoder produces file representations used to assign files to architectural modules.

The investigation starts with file-level dependency graphs in which the signals are supplied as node attributes, examines the signals individually, and then adds two further node types, with member nodes used only for mapping. A folder node represents a directory and is connected to its parent folder and to the files it contains. A member node aggregates all program elements of a single source file, such as its methods and fields, and is connected to that file. Adding these nodes only after the signals have been examined individually makes it possible to separate the effect of the initial file representation from the effects of graph structure and of the way information is shared between connected files. The aim is not to identify one graph model or representation that is uniformly preferable, but to examine under which conditions different signals and learning mechanisms are useful.

The thesis assigns source-code files to architectural modules and evaluates the assignments against a published reference decomposition. The file is the unit of analysis throughout. Member-level information may serve as evidence, but assignments are not made at the class, method, or subsystem level. The thesis spans unsupervised recovery and supervised mapping with limited labeled data and considers static implementation artifacts: dependencies, folder organization, and source code content. Other sources of evidence, such as commit co-change history and dynamic dependencies observed at runtime, are not studied. Chapter 3 states the formal problem and the objectives.

1.4 Thesis Contributions

The thesis makes three research contributions, corresponding to the objectives in Chapter 3, and one methodological contribution. The contributions are based on

Papers I–V, introduced in Chapter 4, and supported by the findings presented in Chapters 6 and 7.

1. **Graph-based representation learning for recovery and mapping.** The thesis develops and evaluates graph-based methods for unsupervised SAR (Papers I, II, and IV) and for C2A mapping from a small labeled set (Papers III and V). It examines alternative learning objectives (Papers I and II), encoder architectures (Papers II–V), and dependency representations (Papers III and IV), and compares the methods with established approaches (Papers I–IV).

2. **Empirical evaluation of representations of implementation signals.** The thesis examines the architectural information carried by dependency structure and by lexical and contextual source-code representations, assessed through their use in recovery and mapping. It evaluates dependency-based recovery (Paper IV) and compares initial file representations for mapping without additional information from message passing (Paper V). It also investigates whether dependency information can adapt contextual code representations to better reflect architectural proximity (Paper V).

3. **Analysis of how graph structure interacts with implementation signals.** The thesis examines whether folder organization (Papers IV and V), member-level detail (Paper V), edge-level attributes (Paper I), and dependency-based message passing (Papers III and V) contribute beyond the initial file representation. The analyses cover recovery and mapping and, where possible, hold the file representation fixed to distinguish the contribution of added structure from that of the representation.

4. **Controlled cross-study evaluation.** The thesis extends the evaluation of recovery and mapping methods to common benchmarks under controlled conditions (Section 5.6). Recovery methods from Papers I, II, and IV are compared on fourteen systems using a common clustering algorithm, reference-decomposition agreement, and dependency cohesion. Mapping methods from Papers III and V are compared on ten systems using matched input representations, initial labeled sets, and training procedures. Each method is run 100 times per system, and results are reported per system with significance tests and effect sizes.

Together, the first three contributions treat model architecture, input representation, graph construction, and system structure as connected design decisions rather than independent sources of improvement.

1.5 Thesis Outline

The remainder of the thesis is organized as follows. Chapter 2 reviews prior work on software architecture recovery and C2A mapping, introduces the structural, organizational, lexical, and contextual representations used in the thesis,

and presents the relevant graph representation learning and evaluation concepts. Chapter 3 defines the research problem, objectives, and questions.

Chapter 4 relates the included studies to research objectives and summarizes their approaches, benchmark scope, main baselines, and principal findings. Chapter 5 presents the shared empirical and constructive research design, study progression, investigated design space, benchmark systems, and evaluation protocol, including the controlled cross-study evaluation.

Chapter 6 synthesizes the findings of the included studies and the cross-study evaluation by research objective. Chapter 7 discusses the findings across studies and addresses their implications and the limitations of the work. Chapter 8 summarizes the main findings and contributions and identifies directions for further research.

Chapter 2

Background

Contents

2.1 Prior Approaches 7
2.1.1 Software Architecture Recovery 9
2.1.2 Code-to-Architecture Mapping 10
2.1.3 Limitations Across Recovery and Mapping 11
2.2 Representing Artifact Content 11
2.2.1 Lexical Representations 12
2.2.2 Contextual Representations 12
2.3 Graph Representation Learning 12
2.3.1 Random-Walk Embeddings 13
2.3.2 Graph Neural Networks 13
2.3.3 Graph Properties and Learning 14
2.3.4 Learning Paradigms 15
2.4 Evaluation and Ground Truth 16

This chapter presents the background needed to frame the research problem and the methods used in this thesis. It first reviews prior approaches to architecture recovery and C2A mapping, with attention to how they represent and combine implementation-level evidence. It then introduces artifact representations, graph-based learning methods and relevant graph properties, learning paradigms, and evaluation concepts used in subsequent chapters.

2.1 Prior Approaches

Prior approaches to architecture recovery and C2A mapping draw on implementation signals such as static dependencies, directory layout, and identifiers and comments in the code. Some also use version history or other metadata, which are not studied in this thesis. The approaches differ in how they represent, combine, and learn from these signals. Table 2.1 summarizes representative approaches, their input signals, underlying methods, and main limitations.

Table 2.1: Representative approaches to SAR and C2A mapping. SD: static dependencies; F: folder hierarchy or file paths; T: identifiers, comments, or other code text. † denotes an additional signal not studied in this thesis.

Approach	SD	F	T	Paradigm	Primary limitation
Software Architecture Recovery					
Dependency-based clustering					
ACDC [80]	✓	×	×	Pattern-driven clustering	Over-fragments into many small clusters
Bunch [54]	✓	×	×	Local search (hill climbing variants); optimizes MQ	Hubs and utility classes distort modules; slow on large systems
DAGC [69]	✓	×	×	Global search (genetic algorithm); optimizes MQ	MQ objective poorly matches expert decompositions
FCA [79]	✓	×	×	Fast clustering on the neighborhood degree matrix	Greedy and local; weights all dependencies equally
LIMBO [3]	✓	×	×	Information-bottleneck hierarchical clustering	Preset cluster count; does not scale to large systems
WCA [51]	✓	×	×	Weighted hierarchical clustering	Preset cluster count; low component-level accuracy
Semantic and name-based clustering					
Anquetil et al. [4]	×	×	✓	File-name clustering using common substrings and lexical concepts	Short or cryptic names can hinder concept extraction
ARC [22]	✓	×	✓	LDA-based topic modeling	Shared terminology mixes modules; preset cluster count
ZBR [12]	×	×	✓	TF-IDF weighted lexical zones, hierarchical clustering	Shared terminology mixes modules
Hybrid multi-signal methods					
SADE [67]	✓	×	✓	Community detection on weighted graph	Predefined, manually tuned weights
SARIF [87]	✓	✓	✓	Community detection on weighted graph	Predefined, manually tuned relation-type and signal weights
NSGA-III† [55]	✓	×	✓	Multi-objective search over structure; uses change history	Objectives fix the fusion before recovery
Representation learning and LLMs					
NEGAR [8]	✓	×	×	Node2Vec-based random walks, clustered	One dependency type at a time; no node attributes
Amalfitano et al. [2]	×	×	✓	Generative LLMs with iterative prompting	Weak on class relationships and fine-grained patterns
SemArc [88]	✓	×	✓	LLM code interpretation, anchor-based clustering	Scalability; possible benchmark overlap with pretraining data

Table 2.1 continued from previous page

Approach	SD	F	T	Paradigm	Primary limitation
Code-to-Architecture Mapping					
Rule-based and retrieval-based mapping					
Tzerpos and Holt [81]	✓	×	✓	Orphan adoption by naming and structure criteria	Hard-coded patterns
HuGMe [9]	✓	×	×	Dependency-based attraction functions for semi-automatic mapping	Requires an initial partial mapping
IRAttract LSIAttract [5]	×	×	✓	Information-retrieval based attraction functions	No explicit dependency structure
InMap [77]	×	×	✓	Interactive ranking from module descriptions	No dependency structure
Learned mapping					
NBAttract [64]	✓	×	✓	Semi-supervised Naive Bayes	Dependencies enter as per-file features, not graph structure
Pretrained embeddings and LLMs					
Johansson et al. [38, 39]	×	×	✓	Pretrained embeddings with classifiers	No explicit dependency or folder structure
Rukmono et al. [76]	×	×	✓	GPT-4 mapping to a predefined architecture	Scalability as system size grows

2.1.1 Software Architecture Recovery

Traditional recovery methods treat software artifacts as a dependency graph and recover modules through several related paradigms. A first group formulates recovery as search over a modularization objective. Some methods optimize a single modularity criterion that favors high cohesion within modules and low coupling across them [54, 69], while others use multi-objective or evolutionary search to balance several criteria at once [71]. A second group clusters directly on dependency-derived similarities or information-theoretic criteria [61, 60, 3, 51]. ACDC [80] takes a different approach by building clusters from recurring subsystem patterns. A third group focuses on the clustering procedure itself. The fast clustering algorithm (FCA) [79] speeds up clustering for large systems by working on the neighborhood degree matrix; Mohammadi and Izadkhah [56] build clusters from the neighborhoods of artifacts in the dependency graph, and Pourasghar et al. [70] use relationship depth. Overall, graph-based recovery methods can work well when dependencies reflect architectural structure, but their effectiveness depends on which dependency relations the graph includes and how the method uses them.

Semantic approaches use identifiers and comments instead of, or alongside, dependencies, and can outperform dependency-only methods in some systems [19]. ARC [22] uses latent Dirichlet allocation (LDA) to infer concerns from

source text and combines them with structural information. ZBR and Risi et al. derive semantic similarities using lexical zones and latent semantic indexing (LSI), respectively [12, 74], while Kargar et al. [40] build a semantic dependency graph from lexical cues and optimize it by local search. Anquetil et al. [4] take a related but distinct approach by clustering files according to their names, using common substrings and concepts found in routine names or comments. Non-structural signals of this kind can be noisy in isolation, since reused terminology may span modules.

Hybrid approaches use multiple sources to capture complementary evidence. Some integrate structural and semantic relations before hierarchical clustering [53, 73]. Others incorporate semantic information into search-based modularization, and NSGA-III also uses development change history [37, 55]. Puchala et al. [72] combine several clustering results into a consensus solution, while Yang et al. [86] use semantic cues to guide label propagation over the dependency graph. Naim et al. [59] combine structural and contextual information through coordinated clustering, refining the combination through incremental developer feedback. SADE [67] and SARIF [87] combine signals in weighted dependency graphs and apply community detection. SARIF also incorporates directory similarity. These methods bring information together at different stages, from constructing relations and defining search objectives to guiding or combining clustering results. In most cases, the fusion is specified by a designed procedure, consensus rule, or heuristic weighting rather than learned jointly as a representation of the signals.

Representation learning has also been applied to recovery. NEGAR [8] learns node representations from dependency graphs through random walks and clusters the resulting vectors. It applies this pipeline separately to graphs at different implementation levels and does not incorporate non-structural signals.

Another approach uses large language models to infer architectural structures and patterns from source code. SemArc [88] uses LLMs to identify implementation-level and architectural semantics, drawing on canonical architectural patterns as a knowledge base. It also incorporates explicit and implicit dependencies before applying component-as-anchor clustering. Amalfitano et al. [2] evaluate general-purpose LLMs on identifying implementation-level class diagrams, architectural and design patterns, and architectural styles. Their prompts include a self-reflection step to simulate iterative refinement. ArchAgent [66] combines static analysis and adaptive code segmentation with LLM-based synthesis to recover multi-view architectures from large, cross-repository codebases.

2.1.2 Code-to-Architecture Mapping

Tzerpos and Holt [81] introduced orphan adoption, the incremental assignment of unmapped entities to a documented architecture using naming and structural criteria. Christl et al. [9] built on this with HuGMe, a semi-automatic method that extends a partial mapping through dependency-based attraction functions

and leaves uncertain assignments for the architect to review. Bittencourt et al. [5] later introduced the information-retrieval attraction functions IRAttract and LSIAttract, which use lexical similarity between entities and modules. InMap [77] takes an interactive recommendation approach, ranking candidate mappings from natural-language module descriptions. Like HuGMe, it keeps the architect involved, but through interactive recommendations rather than review of uncertain assignments. Together, these approaches rely on predefined criteria, attraction functions, or retrieval-based ranking rather than a jointly learned representation, the mapping analog of the fusion schemes above.

Learned classifiers provide another approach to mapping. Olsson et al. [64] propose NBAttract, a semi-supervised Naive Bayes mapper that combines lexical and dependency information encoded via Concrete Dependency Abstraction (CDA). Florean et al. [18] compare text classifiers and text representations for the same task. An earlier study [33] represents dependencies directly as edges in homogeneous and dependency-type-aware graphs, in contrast to NBAttract's feature-based encoding. It evaluates graph convolutional network (GCN) variants and uses iterative pseudo-labeling to expand a small initial labeled set.

Some mapping studies use pretrained embeddings as classifier inputs, while others prompt general-purpose LLMs to produce mappings. Johansson et al. [39] evaluate pretrained embeddings alongside traditional representations such as TF-IDF as input to method-level classifiers. In follow-up work, Johansson and Caporuscio relate the performance of embedding-based mapping of functions and files to data properties, most consistently class separability [38]. Rukmono et al. [76] use chain-of-thought prompting with GPT-4 to map implementation units to a predefined reference architecture. Both approaches use lexical or semantic information, while typed dependencies remain implicit in their inputs.

2.1.3 Limitations Across Recovery and Mapping

Across recovery and mapping, many of the reviewed approaches rely on a single information source, combine sources through predefined rules, or use a fixed representation of each source within a learned model. Comparing the contributions of graph structure, directory organization, and source-code content requires varying these choices under matched conditions. The studies comprising this thesis examine how these signals can be represented, combined, and propagated for architecture recovery and C2A mapping.

2.2 Representing Artifact Content and Organization

A software system carries architectural evidence in more than one form. Its files and members are connected by static dependencies, but they also have names, occupy locations in the folder hierarchy that reflect how developers group related

functionality [20], and contain identifiers and comments that encode domain-specific knowledge [87]. In a representation learning framework, dependencies can be represented as graph relations, while the content associated with individual artifacts must be encoded numerically for a learning model to operate on. This section introduces the two families of representation used in the thesis: lexical and contextual.

2.2.1 Lexical Representations

Lexical representations treat folder-path components and source-code identifiers as tokens. The simplest representation is a binary bag-of-words (BoW) vector, in which each dimension indicates whether a token is present in the artifact's folder path or source code. Because it allocates one dimension to every vocabulary token, the representation becomes high-dimensional and sparse as the vocabulary grows. Word2Vec (W2V) [52] instead learns a dense vector for each source-code identifier from a corpus of source-code tokens using a skip-gram model.

In skip-gram, each token is used to predict tokens occurring within a fixed context window. The vectors are trained to assign higher scores to observed token–context pairs than to randomly sampled pairs through negative sampling. As a result, tokens that occur in similar contexts receive similar vectors. An artifact's Word2Vec representation is obtained by averaging the vectors of its source-code identifiers, producing a compact, fixed-dimensional encoding.

2.2.2 Contextual Representations

Contextual representations encode tokens in the context of the surrounding code rather than as an unordered collection of identifiers. The same identifier can therefore receive different representations in different contexts, whereas a trained Word2Vec model assigns a fixed vector to each token. Transformer-based encoders use attention to combine information from tokens within the input sequence, allowing each token's representation to reflect how it is used in the code. Through pretraining on large corpora, these encoders learn patterns that can support different code analysis tasks. UniXcoder [27] is one such model, producing contextual token representations that can be pooled into a single vector for a code fragment.

A pretrained code encoder can be used without further training or adapted through fine-tuning. Fine-tuning adjusts its representations using a training objective and examples relevant to the target setting, which can include dependency information.

2.3 Graph-Based Representation Learning

Content encoders process artifacts individually, although their representations can capture dependency information introduced during fine-tuning. Graph-

based representation learning uses the dependencies and containment relations between artifacts explicitly to learn node representations. The approaches used in this thesis fall into two groups: random-walk embeddings, which derive vectors directly from graph neighborhoods, and graph neural networks, which propagate and transform information along graph edges.

2.3.1 Random-Walk Embeddings

Random-walk methods learn node embeddings in two steps. First, they construct a corpus by sampling random walks over the graph, where each walk is a sequence of nodes. Node2Vec (N2V) [26] samples walks over a homogeneous graph, which in this thesis is unweighted and undirected, whereas MetaPath2Vec (M2V) [14] extends this to heterogeneous graphs by constraining walks to follow a predefined sequence of node and relation types. Second, a skip-gram model (see Section 2.2.1) is trained over this corpus, treating the nodes within a fixed context window of a target node as its observed context, so that nodes with similar walk contexts receive similar embeddings. Walk length, context window, and the number of walks per node determine which nodes are treated as similar. These methods learn from walk co-occurrence alone and therefore do not jointly incorporate artifact content.

2.3.2 Graph Neural Networks

Graph neural networks (GNNs) provide a common framework for learning from artifact-level information and graph structure, producing node representations shaped by both. A GNN encoder maps each node, together with its initial representation and graph neighborhood, to a vector through message passing [24]. At each layer, a node aggregates the current representations of its neighbors, combines them with its own, and applies a learned transformation. GNN architectures differ primarily in how they transform and aggregate these messages, and this thesis uses three variants.

The graph convolutional network [43] aggregates neighboring representations through degree-normalized message passing, which helps prevent high-degree nodes from dominating solely because they have many dependencies. The graph attention network (GAT) [82] instead learns attention weights over neighbors, allowing their contributions to differ according to their representations. Both encoders can be applied either to a graph in which dependency types are collapsed or to a multi-relational graph with relation-specific parameters. When the graph contains multiple node types and typed relations, this thesis also uses the heterogeneous graph transformer (HGT) [29], which parameterizes attention by meta-relation. Thus, the contribution of a neighbor depends jointly on the source-node type, target-node type, and edge type.

2.3.3 Graph Properties and Learning

Two properties are particularly relevant to how well graph-based encoders can support downstream tasks: homophily and structural reach. A graph is homophilic with respect to a labeling when connected nodes tend to share a label. Under homophily, neighborhood aggregation reinforces label-consistent information; under heterophily, it mixes information from different labels and may obscure distinctions needed by the downstream task [89]. Formally, let $G = (V, E)$ be a graph in which each node $v \in V$ has a label y_v. Homophily $\mathcal{H}$ is the proportion of edges whose two endpoints have the same label,

$$\mathcal{H} = \frac{|\{e = (u, v) \in E : y_u = y_v\}|}{|E|}, \tag{2.1}$$

so that $\mathcal{H} = 0$ means every edge connects nodes with different labels and $\mathcal{H} = 1$ means every edge connects nodes with the same label [89]. For software systems, the nodes are files, the edges are static dependencies between them merged across dependency types, and the label is the ground-truth module. In this setting, homophily relates to cohesion and coupling by measuring the proportion of dependencies that remain within modules rather than cross module boundaries. It provides an overall measure of this balance, but does not distinguish how those dependencies are distributed across individual modules.

Since $\mathcal{H}$ is computed from the reference decomposition, it can be determined only once such a decomposition exists. The concentration of dependencies within modules varies across systems, and so does the usefulness of dependency-driven random walks and message passing.

The second property is the structural reach of the encoder, which we use to mean the set of nodes that can contribute information to a node representation. For an L-layer message-passing encoder, this is the node's L-hop neighborhood. Reach describes which nodes can contribute by construction, rather than how much they contribute in practice. Increasing depth extends this reach, but repeated aggregation can make node representations progressively less distinguishable, an effect known as over-smoothing [49]. GNN encoders are therefore commonly kept shallow.

With a shallow encoder, access to more distant architectural information depends on how the input graph connects the nodes. Changing these connections can make that information reachable within fewer message-passing steps. Random-walk embeddings can capture longer-range structural context through sampled walks, although they do not propagate artifact-level representations along the graph.

Two further techniques alter the structural information available to a shallow encoder without increasing its number of message-passing layers. Laplacian positional encoding (LPE) augments each node representation with leading eigenvectors of the graph Laplacian, giving the encoder information about the node's structural position before message passing begins [15]. Graph diffusion convolution (GDC) [23] rewires the input graph using personalized PageRank

(PPR), connecting diffusion-close nodes directly and assigning the resulting edges diffusion-derived weights. This makes multi-hop structural proximity available within a fixed number of message-passing layers.

2.3.4 Learning Paradigms

The encoder architecture determines how information propagates through the graph, but not what drives learning. That role belongs to the training objective, which depends on the supervision available. The studies in this thesis use two learning settings that reflect the recovery and mapping tasks.

Architecture recovery. No module labels are available during recovery, so the studies use self-supervised learning, in which the model learns from an auxiliary signal derived from the graph itself. These methods construct positive and negative training examples from the graph, with the examples differing between methods. Binary cross-entropy (BCE) measures how well the model distinguishes positive from negative examples. For a single example, it is defined as:

$$\mathcal{L}_{\mathrm{BCE}} = -\left[y \log \hat{y} + (1-y) \log(1-\hat{y})\right], \tag{2.2}$$

where $y \in \{0, 1\}$ is the target and $\hat{y}$ is the predicted probability.

Contrastive learning trains representations by contrasting positive and negative pairs. Deep Graph Infomax (DGI) [83] applies this principle to graphs by treating node–summary pairs in the original graph as positive and pairs constructed from a corrupted view as negative. Graph autoencoders (GAE) [44] instead learn node representations by reconstructing observed graph edges, treating observed edges as positives and sampled non-edges as negatives. Their variational form (VGAE) [44] models each node's latent representation as a Gaussian distribution rather than a single vector and adds a Kullback–Leibler (KL) divergence term that penalizes deviations from a standard normal prior. Random-walk methods learn without module labels through the skip-gram objective described in Section 2.3.1.

The learned embeddings are then partitioned with a clustering algorithm, typically agglomerative hierarchical clustering [58], for architecture recovery. This clustering step is unsupervised: it derives module groups from similarity between embeddings. The number of clusters is either selected automatically from a predefined search range or fixed in advance.

C2A mapping. Mapping starts from known module assignments for a limited number of files, so the studies train a classifier on these labeled files with supervised learning. Because the module set is known and fixed, the classifier assigns each file to one of the K predefined modules through a softmax layer, which yields a probability distribution over the modules, and the highest softmax probability of a prediction is its confidence. It is trained with categorical cross-entropy (CCE), $-\sum_{k=1}^{K} y_k \log \hat{y}_k$, which for a one-hot target is the negative log-probability assigned to the true module.

Unlabeled files can also contribute through iterative self-training [47], which Paper III applies on top of supervised training. The classifier assigns pseudo-labels to unlabeled files whose confidence exceeds a predefined threshold, retrains on the labeled and pseudo-labeled files, and repeats over successive rounds until no prediction meets the criterion or a maximum number of rounds is reached.

2.4 Evaluation and Ground Truth

The studies in this thesis evaluate their outputs against reference decompositions from published, expert-validated benchmarks, supporting comparison with prior work. Recovery produces a partition whose cluster labels have no predefined meaning, whereas mapping assigns files to known architectural modules. Recovery is therefore evaluated through partition-agreement metrics and mapping through conventional classification metrics. This section introduces these measures and discusses how reference decompositions should be understood as ground truth. The provenance of the specific reference mappings is discussed in Chapter 5.

A reference decomposition provides an established basis for comparison, but it need not be the only reasonable architectural interpretation of a system. Experts may disagree on architectural boundaries and design intent [20], so a recovered decomposition can differ from the reference without necessarily being wrong.

Complementary metrics assess different aspects of a recovered decomposition. MoJoFM, A2A, and $C2C_{cvg}$ measure alignment with a reference decomposition. The adjusted Rand index (ARI) [30] measures partition agreement while correcting for chance. TurboMQ provides a complementary, reference-independent measure of modularization quality based on the dependency graph.

MoJoFM [84] is widely used in architecture recovery. It counts the minimum number of move and join operations needed to transform the recovered partition into the reference partition, normalized by the maximum possible number of such operations. Because its join operations are comparatively cheap, MoJoFM is less sensitive to over-fragmentation: many small clusters can still score well when they broadly align with the reference. A2A [46] extends MoJoFM with costs for adding, removing, and moving entities and clusters. It can assign a high score to two architectures that are structurally quite different, overestimating their similarity [50].

$C2C_{cvg}$ [21] measures the percentage of reference modules that are sufficiently covered by a recovered module, where coverage is judged against a threshold on the overlap between the two; conventional thresholds of 0.10, 0.33, and 0.50 correspond to weak, moderate, and majority matches. This makes it sensitive to fragmentation: a reference module split across several recovered modules does not count as matched if none of them meets the chosen overlap threshold, even where individual file assignments look reasonable under a different metric. ARI ranges from -1 to 1, where 0 is the agreement expected by chance.

Unlike the metrics above, which measure alignment with the reference decomposition, TurboMQ [50] measures modularity: it rewards intra-cluster cohesion and penalizes inter-cluster coupling, computed directly on the dependency graph without reference to any external partition. TurboMQ can favor decompositions with many clusters. Normalized TurboMQ (TMQ_{norm}) divides the TurboMQ score by the number of recovered clusters to reduce this bias.

The reference-agreement metrics described above assess how closely a recovered decomposition matches an expert-produced reference, whereas TurboMQ evaluates its dependency structure. Expert-produced references can reflect responsibilities and project organization identified through documentation, source code, and knowledge of the system. These boundaries need not coincide with groups of densely connected files, since shared utilities and libraries can create dependencies across architectural modules [20]. A reference decomposition therefore need not maximize dependency modularity, and a recovered decomposition can score well on one kind of measure without scoring well on the other.

For mapping, the evaluation is a per-file classification into known modules, and the standard metrics apply directly. Precision, recall, and their harmonic mean F1 are reported per module and aggregated at the macro and micro level. The macro-average treats every module equally and therefore surfaces performance on small modules that would be hidden by a micro-average dominated by large ones. Reporting both reveals differences in performance across modules of different sizes.

Chapter 3

Problem Formulation and Research Objectives

Contents

3.1 Problem Formulation 19
3.2 Research Objectives 20
3.2.1 Representation Learning for Recovery and Mapping 21
3.2.2 Representations of Implementation Signals 21
3.2.3 Implementation Signals and Graph Structure 22

This chapter formalizes the research problem addressed in this thesis and defines the objectives for investigating it. It first presents software architecture recovery and C2A mapping as related file-to-module assignment problems within a common graph-based representation-learning framework. It then states the research objectives and associated research questions.

3.1 Problem Formulation

Architecture recovery and C2A mapping both assign source-code files to architectural modules, but differ in the architectural information available. This thesis approaches recovery by learning file representations without module labels and clustering them into modules. In C2A mapping, the target modules and a subset of file assignments are known, and the task is to predict the remaining assignments. Both tasks are formulated here using a common graph representation.

Despite this difference in supervision, both tasks draw on dependencies, project organization, and source-code content as discussed in Section 1.1. This thesis uses graph-based representation learning to combine these signals as evidence of module membership.

Accordingly, a software system is represented as a graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R}, \mathrm{X}), \tag{3.1}$$

where $\mathcal{V}$ and $\mathcal{E}$ are the node and edge sets, $\mathcal{A}$ and $\mathcal{R}$ are the node and relation types, and X denotes node attributes. The maps $\tau : \mathcal{V} \to \mathcal{A}$ and $\phi : \mathcal{E} \to \mathcal{R}$

assign types to nodes and edges. Structure-only settings use no lexical or folder attributes. File nodes form the subset $\mathcal{V}_{\texttt{file}} \subseteq \mathcal{V}$.

The graph is homogeneous when $|\mathcal{A}| = |\mathcal{R}| = 1$, and heterogeneous otherwise. Its simplest form is a software dependency graph (SDG), containing only file nodes and a single dependency relation type. Preserving dependency types increases $|\mathcal{R}|$, while adding folder and member nodes increases both $|\mathcal{A}|$ and $|\mathcal{R}|$. Folder and member nodes provide context for learning file representations; only file nodes are assigned to architectural modules.

Formally, for recovery, we learn an encoder f_θ in a self-supervised manner:

$$f_\theta : \mathcal{G} \longrightarrow \mathrm{Z} \in \mathbb{R}^{|\mathcal{V}_{\texttt{file}}| \times d}, \tag{3.2}$$

that maps each source code file $i \in \mathcal{V}_{\texttt{file}}$ to a d-dimensional embedding $\mathrm{z}_i \in \mathbb{R}^d$, where θ denotes the learnable parameters. The resulting file embeddings are clustered into a recovered architectural decomposition $\hat{\mathcal{C}}$, which partitions the source code files into m disjoint modules:

$$\hat{\mathcal{C}} = \{C_1, \ldots, C_m\}. \tag{3.3}$$

In C2A mapping, a set of target modules $\mathcal{C} = \{C_1, \ldots, C_m\}$ is known, together with a labeled subset of files $\mathcal{L} \subseteq \mathcal{V}_{\texttt{file}}$ and their module assignments $Y_{\mathcal{L}}$. The task is to learn:

$$f_\theta : (\mathcal{G},\ \mathcal{L},\ Y_{\mathcal{L}}) \longrightarrow \hat{Y}_{\mathcal{U}}, \tag{3.4}$$

for the unlabeled files $\mathcal{U} = \mathcal{V}_{\texttt{file}} \setminus \mathcal{L}$. The model is trained on the labeled files, and the supervision is limited to this subset. Training may additionally use the model's confident predictions on unlabeled files as pseudo-labels. The setting is transductive: all files, labeled and unlabeled, are part of $\mathcal{G}$ during training.

The two problems use the same implementation signals, but differ in how representations are learned and how their contribution can be assessed. In recovery, module labels are unavailable, so representations are shaped by the graph construction, the input representations, and a self-supervised objective. The final decomposition also depends on the clustering procedure and granularity. In C2A mapping, known file-to-module assignments provide task-specific supervision, allowing matched configurations to be compared using per-file classification metrics. Mapping thus provides a complementary setting for examining whether individual signals help predict an existing decomposition. However, its results depend on the number of initially labeled files, their coverage and distribution across modules, and the training procedure. A signal's usefulness or ranking in one task therefore does not necessarily carry over to the other.

3.2 Research Objectives

The research aim is operationalized through three objectives that progress from joint representation learning to examining individual signals and file representations, and then to their interaction within richer graphs. Objective 1 designs,

compares, and benchmarks graph-based methods for recovery and mapping by combining dependency, organizational, and lexical signals. This examines their joint usefulness, but not their individual contributions. Objective 2 therefore examines the architectural information these signals and representations carry, including how source-code content should be represented. Objective 3 investigates how their usefulness changes as structural context is added and whether findings from Objectives 1 and 2 remain valid.

3.2.1 Representation Learning for Architecture Recovery and C2A Mapping

Objective 1

To design, compare, and benchmark graph-based representation learning methods for architecture recovery and code-to-architecture mapping.

RQ1.1 How can implementation-level signals be combined into a joint representation for architecture recovery and mapping?

RQ1.2 How does modeling dependency types as separate relations compare with merging them into a single relation?

RQ1.3 How sensitive is the learned representation to encoder architecture, learning objective, and model depth?

This objective is addressed through comparisons across recovery and mapping studies on file-level graphs. Most comparisons jointly encode folder, lexical, and dependency signals within one encoder. The comparisons cover GCN, GAT, and HGT encoders and merged versus relation-specific dependencies. Recovery experiments compare contrastive and reconstruction objectives using the same encoder, inputs, and benchmark systems. Mapping studies also vary model depth; one study and its cross-study extension use inputs without folder signals.

3.2.2 Representations of Implementation Signals

Objective 2

To determine what architectural information dependency structure and individual file representations provide, and how source-code content should be represented.

RQ2.1 How far can dependency structure alone support architecture recovery, in agreement with the reference and in dependency cohesion?

RQ2.2 How does dependency-guided adaptation of a contextual code representation affect the architectural signal it carries?

RQ2.3 How do structural, lexical, and contextual file representations compare in mapping without message passing?

This objective examines the architectural information provided by representations of individual implementation signals. In recovery, it uses dependency structure alone on the homogeneous SDG, without lexical or folder information, and examines how much structural context is needed across systems. In mapping, it compares file representations without adding folder organization, member-level detail, or SDG-based message passing. These representations include Word2Vec, zero-shot UniXcoder, Node2Vec, and UniXcoder fine-tuned through SDG-guided contrastive learning without module labels. The latter two incorporate dependency information during representation learning. A multilayer perceptron (MLP) classifier uses each file's representation independently, so it can access dependency information only if that information is already encoded in the representation.

3.2.3 Interaction Between Implementation Signals and Graph Structure

Objective 3

To determine how the contribution of file representations changes with dependency-based message passing, folder organization, and member-level graph detail, and whether findings from Objectives 1 and 2 remain valid.

RQ3.1 How does the usefulness of message passing over dependency neighbors vary across predictors, aggregation directions, and file representations?

RQ3.2 How does adding folder organization to the graph change what the file representations contribute to recovery and mapping?

RQ3.3 How does performance differ with and without member-level nodes and edge-level interaction details?

This objective examines how additional graph structure changes the contribution of file representations. In recovery, folder nodes and containment meta-relations are added to the SDG and evaluated through message passing or random walks; edge attributes describing the member-level interactions behind each dependency, including their type and frequency, are examined separately. In mapping, an MLP and graph-based neural predictors are compared, and each predictor is evaluated with and without access to dependency neighbors while holding file representations fixed. Aggregation direction is also varied. Folder and member nodes are evaluated with Word2Vec, Node2Vec, and fine-tuned UniXcoder. Member nodes also change how lexical and contextual content enters the model, and a rule based on the directory hierarchy and initial labels provides a reference.

Chapter 4

Overview of the Studies

Contents

4.1 Relationship Between Objectives and Studies 23
4.2 Overview of the Included Studies 23
4.2.1 Paper I: Software Architecture Recovery 24
4.2.2 Paper II: Software Architecture Recovery 25
4.2.3 Paper III: Code-to-Architecture Mapping 25
4.2.4 Paper IV: Software Architecture Recovery 25
4.2.5 Paper V: Code-to-Architecture Mapping 26

This chapter introduces the five studies that constitute this thesis and explains how they address the research objectives. It first maps the studies to the objectives, then summarizes what each study built, its benchmark scope, its main baselines, and its principal individual findings.

4.1 Relationship Between Objectives and Studies

The objectives are addressed across complementary studies rather than by a single experimental design applied identically to both recovery and mapping. Figure 4.1 maps each of the five studies to the objectives it addresses, distinguishing studies that directly address an objective's research questions from studies that partially inform it through a contrasting result or an earlier comparison with a narrower scope.

4.2 Overview of the Included Studies

The studies progress from relation-aware graph representation learning for architecture recovery to dependency-based message passing for C2A mapping, then examine structure-only recovery and systematically compare and combine implementation signals for mapping.

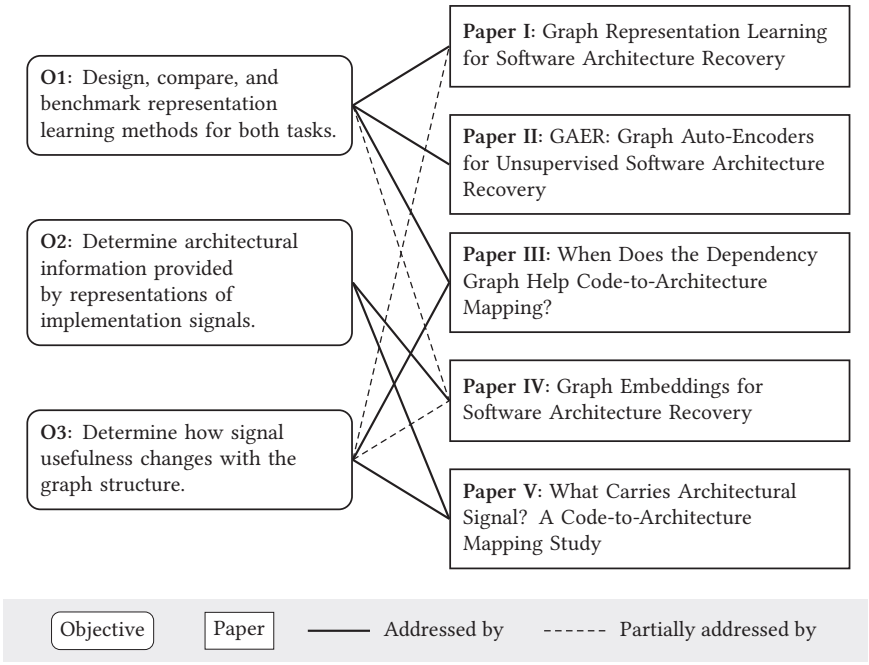


Figure 4.1: Mapping of research objectives to the five studies. Solid lines indicate direct coverage of the research questions; dashed lines indicate partial coverage through contrasting results or narrower comparisons.

4.2.1 Paper I. Graph Representation Learning for Software Architecture Recovery

The first study [35] establishes the initial feasibility of learning architectural representations from a multi-relational software dependency graph for architecture recovery. Each dependency type is modeled as a separate directed relation among source-code files, while node features combine folder-hierarchy tokens with code identifiers not involved in inter-file dependencies. Relation-specific GAT encoders produce file representations that are trained without architectural labels using a contrastive objective and subsequently clustered to recover modules. The study also examines whether member-level interaction attributes add architectural information beyond the presence of a dependency.

The study evaluates the proposed approach on four systems against SARIF [87], a hand-tuned information-fusion baseline that combines dependency, textual, and directory signals through manually designed similarity functions. The proposed approach reaches higher MoJoFM than SARIF on all four systems.

4.2.2 Paper II. GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery

The second study [32] extends the investigation of Paper I to a reconstruction-based learning approach and a broader empirical setting. GAER retains the multi-relational dependency graph but replaces contrastive learning with a graph autoencoder trained to reconstruct observed dependencies. Code identifiers are now embedded with Word2Vec rather than encoded as binary vectors, while directory tokens remain binary-encoded. The study compares GCN and GAT encoders before clustering the learned representations into architectural modules.

The study evaluates ten benchmark systems against eight established dependency-based, semantic, structural representation learning, and hybrid recovery approaches. GAER achieves higher or comparable ground-truth alignment across most systems for MoJoFM, A2A, $C2C_{cvg}$, and ARI.

4.2.3 Paper III. When Does the Dependency Graph Help Code-to-Architecture Mapping?

The third study [36] is an extended version of an earlier mapping paper [33]. It shifts from unsupervised architecture recovery to C2A mapping, in which the target modules are known, and a limited set of files is initially labeled. It reuses the graph construction and node features of Paper II, expanding the labeled set into a full mapping via iterative self-training. The study compares GCN and GAT models with an MLP that uses the same file features, classifier, and self-training procedure. It further examines the effect of preserving dependency types, message-passing depth, and aggregation direction.

Across ten benchmark systems, all learned approaches outperform the Naive Bayes mapping baseline [64]. The gain of GAT and GCN over the MLP varies by system and is largest on Chromium, the largest system.

4.2.4 Paper IV. Graph Embeddings for Software Architecture Recovery

The fourth study returns to architecture recovery but isolates the value of structural and organizational information by removing lexical file features [34]. Unlike the first three studies, it learns representations from graph structure alone. The study first evaluates recovery from a homogeneous dependency graph using Node2Vec, a GAT encoder trained within a variational graph autoencoder, and the same encoder applied to a graph rewired by PPR diffusion (GAT-DC). It then extends the graph with folder nodes and containment relations, comparing the dependency structure alone against the dependency structure combined with developer-defined folder organization, using MetaPath2Vec, a heterogeneous GAT (HGAT), and a variant that preserves dependency types rather than merging them (HGAT-TD).

The study evaluates eleven systems against four classical dependency-only recovery baselines. The dependency-only GAT-DC and Node2Vec outperform these baselines on most systems and metrics.

4.2.5 Paper V. What Carries Architectural Signal? A Code-to-Architecture Mapping Study

The fifth study examines which implementation signals contribute to C2A mapping and how their usefulness changes with graph structure and message passing [31]. It compares structural Node2Vec, lexical Word2Vec, and contextual UniXcoder file representations, with UniXcoder evaluated before and after dependency-guided contrastive fine-tuning. Each representation is first evaluated using an MLP. Node2Vec, Word2Vec, and fine-tuned UniXcoder then initialize HGT on four graph configurations: files only, files with folders, files with members, and the full folder-file-member graph, with dependencies merged into a single relation type. Each member node summarizes the program members of one file. A GAT and an MLP that receives the mean of its dependency neighbors' vectors are evaluated on the file-level dependency graph as further predictors, with dependency neighbors enabled or disabled and aggregated in one or both directions. A model that supplies folder-path tokens directly in the file vector is compared with folder nodes, and HGT is also evaluated with two layers.

The study uses the ten systems and labeled sets of Paper III. The MLP is the baseline for the graph configurations, and a directory-hierarchy rule is the reference for folder-augmented configurations, whether through folder nodes or folder-path tokens supplied in the file vector. The rule assigns an unlabeled file the most common module among the labeled files in its directory, backing off to progressively larger enclosing subtrees when the directory has no labeled file associated with it.

Chapter 5

Research Method

Contents

5.1 Research Design 27
5.1.1 Design Space 28
5.1.2 Research Trajectory 28
5.2 Graph and Signal Representation 30
5.2.1 Graph Construction 30
5.2.2 Node Feature Initialization 31
5.2.3 Edge Attributes 32
5.3 Learning Settings 32
5.3.1 Controlled-Comparison Logic 34
5.4 Benchmark Systems 35
5.5 Evaluation Protocol 36
5.5.1 Labeled Training Sets 36
5.5.2 Statistical Analysis 37
5.6 Cross-Study Evaluation Design 37
5.6.1 Cross-Study Correlation with System Properties 38

This chapter describes the research method used to investigate architecture recovery and C2A mapping. It defines the research design and the graph-based representation-learning design space, then describes the input signals, learning settings, and controlled-comparison logic used across the studies. It also presents the benchmark systems and evaluation protocol, including the sampling of labeled files and statistical analysis. Finally, it describes the additional cross-study evaluation design.

5.1 Research Design

This thesis combines method development with empirical evaluation. Each study develops and evaluates a graph-based representation learning method for architecture recovery or C2A mapping. The studies investigate what architectural information is carried by implementation signals, including dependency structure, project organization, and code content, and the system conditions under which

these signals support architectural analysis. Comparisons with established baselines assess whether the resulting methods are competitive, whereas controlled comparisons between graph configurations, input representations, and learning mechanisms examine the contribution of each.

5.1.1 Design Space

Architecture recovery and mapping are studied through a common graph-based representation-learning framework. The choices investigated across the thesis define a design space with four dimensions: graph topology, relation modeling, input representation, and learning mechanism, summarized in Figure 5.1. These dimensions are analytically distinct but not always experimentally independent: adding a node type usually requires a new relation type to connect it to the rest of the graph.

Graph topology determines which software artifacts are represented as nodes. The simplest configuration contains only file nodes. Richer variants add folder nodes, one per directory, member nodes that aggregate the program elements contained in each file, or include both folder and member nodes together. Relation modeling determines how these nodes are connected. Static dependencies may be merged into a single relation or preserved as typed relations such as *call*, *import*, and *inheritance*. Graphs containing additional artifact types also include organizational and containment meta-relations, such as folder-to-folder, folder-to-file, and file-to-member relations.

Input representation determines what information is available to the nodes before message passing. Nodes may start with learnable identity embeddings or with features derived from dependency structure, folder organization, or code content. These features range from token-based representations and structural encoding to pretrained contextual code embeddings.

The learning mechanism determines how these inputs and graph relations are used to learn file representations. The studies compare random-walk methods and graph neural networks, with learning objectives suited to recovery or mapping.

5.1.2 Research Trajectory

The studies progressively examine how dependency structure, project organization, and code content contribute to recovery and mapping. Each study addresses a question left open by earlier work and informs the choices investigated in the next. Together, they cover an expanding range of graph configurations, input representations, and learning mechanisms.

The prior approach, NEGAR [8], evaluates dependency types through separate pipelines and uses neither folder organization nor lexical information, leaving open whether typed dependencies can be modeled jointly and whether organizational and lexical signals provide complementary evidence for recovery. Papers I and II take a first step on this question with a multi-relational file-level

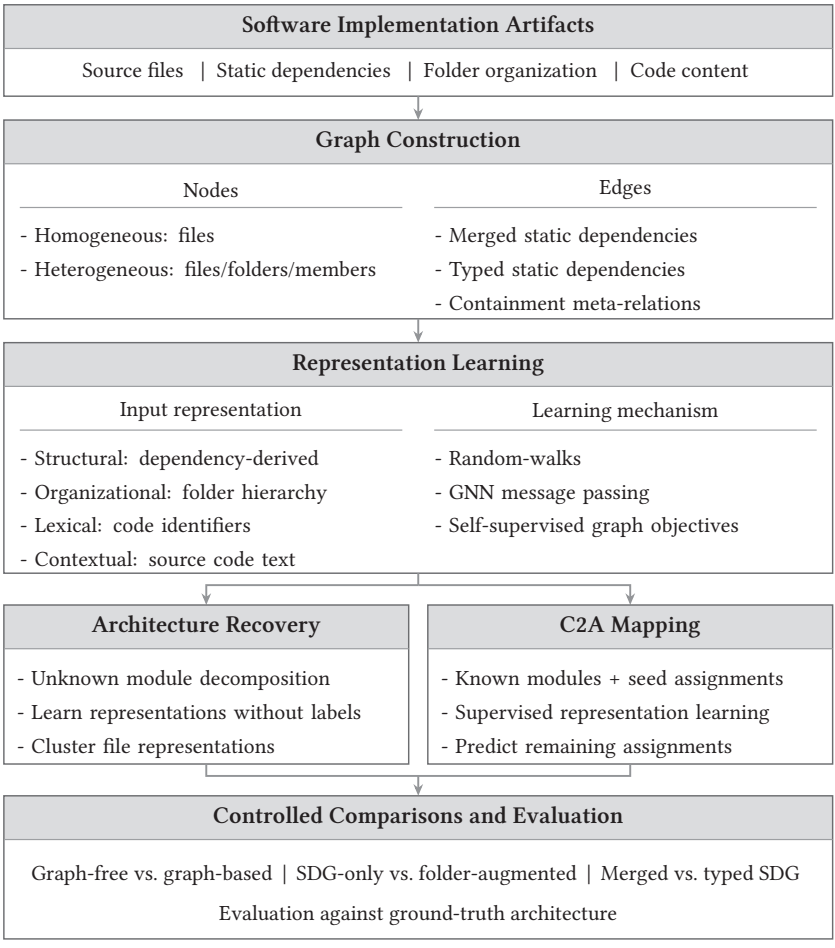


Figure 5.1: Design space studied across the thesis: graph configurations ranging from the SDG to heterogeneous folder-file-member graphs, initialized from structural, organizational, lexical, or contextual signals, and evaluated under unsupervised recovery and supervised mapping through controlled comparisons of relation modeling, graph structure, input representation, and learning mechanism.

graph. Both fuse folder-path and code-identifier tokens as node features, so the contribution of each source cannot yet be separated from the others.

Paper III keeps the graph and feature construction of Paper II, apart from the Word2Vec settings, and moves to a semi-supervised setting, formulating mapping as node classification. It introduces an MLP baseline, trained under the same features and self-training procedure, to compare GNN-based prediction against feature-only prediction.

Paper IV, a further recovery study, returns to dependency structure and project organization, without lexical features. Removing the lexical features allows the contribution of project organization to be assessed without the fused signal of the earlier studies. It compares dependency-only file graphs against folder-augmented heterogeneous graphs under matched conditions, using both random-walk and graph-attention learners, and evaluates ways to extend the effective reach of message passing.

Paper V extends the mapping design by introducing a dependency-guided contextual code representation and by expanding the graph to include folder and member nodes. It compares structural, lexical, and contextual representations with and without message passing, and across graph variants that add or remove folder and member nodes, under one matched supervised mapping setting.

5.2 Graph and Signal Representation

The design space defines the representational choices in the abstract. This section instantiates them concretely, showing which graph each study constructs and which signals it places on that graph's nodes and edges. Table 5.1 summarizes these choices across the five studies.

5.2.1 Graph Construction

Across the five studies, graph topology evolves from file-level dependency graphs to representations that additionally include folder and member nodes. Files remain the targets of recovery or mapping, whereas folder and member nodes provide contextual information through message passing. Static dependencies between code entities are extracted with the multilingual analysis tool Depends [62] and aggregated to the file level. Figure 5.2 illustrates the node-set configurations used in the thesis.

Papers I, II, and III use the file-only configuration, with static dependencies kept as separate typed relations. Paper III further tests a merged-relation variant to determine whether relation-specific message passing contributes to mapping.

Papers IV and V add context through folder nodes, and Paper V also through member nodes. Both merge dependencies into a single relation type. Paper IV compares the file-only graph with a folder-augmented configuration to isolate the contribution of project organization, with a typed-dependency variant as an ablation within the folder-augmented graph. Paper V adds summarized member nodes, each representing the program elements contained in one file, and evaluates four graph variants: file only, file with folders, file with members, and the complete file–folder–member graph. In the complete graph, dependencies keep their file- or member-level endpoints; when member nodes are removed, member endpoints are projected onto their containing files.

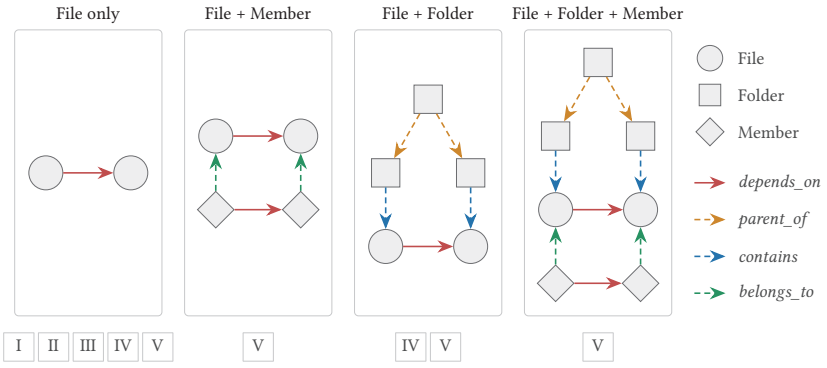


Figure 5.2: Node-set configurations used across the thesis, mapped to the studies that report results on each: file only (Papers I–V), file with folders (Papers IV and V), file with members, and file with folders and members (Paper V). Files are the prediction targets in all configurations. Solid *depends_on* edges are static software dependencies, such as *call*, *import*, and *inheritance*. Dashed edges are contextual meta-relations: folder–folder *parent_of*, folder–file *contains*, and member–file *belongs_to*.

5.2.2 Node Feature Initialization

Node-feature initialization operationalizes the input-representation dimension of the design space. Papers II and III initialize each file node by concatenating a binary BoW representation of its folder path with the ℓ_2-normalized mean of Word2Vec embeddings for its code identifiers. Paper I uses the same sources but represents both folder path and identifier tokens as ℓ_2-normalized binary BoW vectors. Organizational and lexical information therefore enter these models as a fixed input.

Paper IV uses no external lexical or organizational features. Instead, each node type receives a learnable identity embedding trained jointly with the model; file nodes additionally receive LPE, which supplies structural position in the dependency graph. Its Node2Vec and MetaPath2Vec baselines use no input feature vectors, since they learn representations directly from random-walk co-occurrence.

Paper V treats node initialization as the object of study rather than a fixed choice. File and member nodes use lexical Word2Vec embeddings over identifiers, structural Node2Vec embeddings from the dependency graph, or contextual UniXcoder embeddings from code text extracted from source files, evaluated in zero-shot (UXC_{ZS}) and dependency-guided fine-tuned (UXC_{FT}) forms. Where folder nodes are included, they are consistently initialized using binary BoW vectors derived from folder-path tokens. Paper V therefore varies the representation of file- and member-level information while holding folder-node initialization constant.

Input text is extracted using Tree-sitter [7] with the Java, C, or C++ grammar matching each file. The syntax tree guides fragment selection but is not itself passed to the encoder. A file-level view contains type declarations, fields or global variables, and constructor signatures with short body excerpts where applicable. A member-level view contains method and function declarations and definitions. Each file receives one vector per view. The file-level view is encoded as a single input, while the member-level vector is obtained by mean-pooling representations of overlapping text chunks.

In zero-shot use, UniXcoder encodes these views without further training. It is also fine-tuned separately for each system on file-level views using a contrastive objective [65] to incorporate dependency structure. Positive pairs are drawn from each file's two-hop dependency neighborhood, falling back to a direct neighbor when no two-hop candidate exists. Other files in the batch serve as negatives, except directly connected files, which are masked unless selected as the positive pair. Module labels are not used in this loss. The resulting model encodes both views, although fine-tuning uses only file-level text.

5.2.3 Edge Attributes

Edge attributes form a narrower axis of the design space and are used beyond relation type in only Papers I and IV. In the remaining studies, edges are defined only by their relation type. Paper I evaluates a variant that augments file-level dependency edges with attributes describing the member-level interactions that produce them: an ℓ_2-normalized binary encoding of interaction type and a log-normalized interaction frequency. This tests whether the origin and frequency of a dependency provide information beyond its typed presence.

Paper IV uses edge attributes differently in its diffusion variant. GDC rewires the file graph using PPR and assigns each rewired edge a scalar diffusion weight, which biases attention towards diffusion-close neighbors. Unlike the attributes of Paper I, these weights are derived from graph topology rather than from additional implementation information. They therefore extend the structural reach of the encoder without adding information beyond the dependency graph.

5.3 Learning Settings

Once the graph and node features are constructed, the studies differ in how they learn file representations and how these representations are used to obtain module assignments. This difference follows the two tasks: recovery methods learn embeddings and cluster them into modules, whereas mapping methods predict assignments to a predefined module set. Table 5.2 summarizes the encoder, learning objective, and training configuration used in each study.

For multi-relational graphs, GCN and GAT use separate layers for each dependency type and aggregate their outputs. HGT instead incorporates node and

Table 5.1: Graph construction and signal representation across the five papers (P). Numbers in parentheses give vector dimensions; ID denotes code identifiers.

P	Nodes	Edges	Node features	Edge features
I	File	typed	[BoW(*path*) ‖ BoW(ID)]	type & frequency†
II	File	typed	[BoW(*path*) ‖ W2V(ID) (100)]	–
III	File	typed / merged†	[BoW(*path*) ‖ W2V(ID) (128)]	–
IV	File / +Folder†	merged / typed†	Folder: Learnable (128) File: [Learnable (128) ‖ LPE (16)]	PPR rewired edges & diffusion weights†
V	File / +Folder / +Member	merged	Folder: BoW(*path*) W2V (128) / N2V (128) / UXC_{ZS} / UXC_{FT} (768)	–

† denotes an ablation against the other value, not the fixed design of the study.

relation types directly in its attention mechanism, processing the relation types jointly within each layer.

In the recovery studies (Papers I, II, and IV), no module labels are available during training. Paper I learns file embeddings using a GAT with a contrastive objective, while Paper II compares GCN and GAT encoders trained through graph reconstruction against a Node2Vec baseline that follows NEGAR [8]. Paper IV compares random-walk methods with GAT-based encoders trained within a variational graph autoencoder. Its graph-attention variants examine diffusion, folder context, and typed dependencies.

The learned file embeddings are then clustered into modules using agglomerative hierarchical clustering with Ward linkage, except for the Node2Vec embeddings of Papers II and IV, which use k-means, following NEGAR [8]. The number of clusters is either selected automatically or set to the number of ground-truth modules, as in the main evaluation of Paper IV. Recovery therefore infers the file assignments and, unless the number of clusters is fixed, also the number of modules.

In the mapping studies (Papers III and V), the target modules are known, and a small set of files is labeled with their ground-truth assignments. Paper III compares GCN, GAT, and MLP models under the same feature representation and training procedure, and uses iterative self-training to expand the labeled set. Paper V compares MLP, GAT, and HGT. The baseline MLP uses file features alone, GAT uses the file-only graph, and HGT is evaluated on all four graph variants. Additional comparisons examine dependency-neighbor aggregation, including a neighbor MLP that receives each file's own vector concatenated with the mean vector of its dependency neighbors. Paper V trains each model on the labeled files for 200 epochs without self-training. It uses the same loss function, optimizer, and learning rate as Paper III, with encoder configurations given in Table 5.2. Mapping therefore predicts file assignments to a predefined module set.

Table 5.2: Encoder configurations and training settings across the five studies, grouped by architecture recovery and C2A mapping. Where a study evaluates multiple encoders, each encoder is listed separately. Dim. denotes the encoder embedding dimension; LR denotes learning rate. Dataset-specific settings for the largest systems are omitted and reported in the individual papers.

	P	Encoder	Objective	Dim.	Layers	Heads	Optimizer LR	Epochs
Recovery	I	GAT	Contrastive	128	1	1	Adam [42] 10^{-3}	50
		GAT-E		128	1	1		
	II	GCN	Reconstruction	128	1	–	Adam 10^{-4} $(10^{-3})^{\dagger}$	30 (50)†
		GAT		128	1	1		
	IV	N2V	Skip-gram	128	–	–	SGD [25] 0.025	5
		M2V		64	–	–	SparseAdam 0.05	20
		GAT	Reconstruction + KL	256	2	2	Adam 10^{-3}	100
		GAT-DC		256	1 or 2‡	2		
		HGAT		256	1	2		
		HGAT-TD		256	1	2		
Mapping	III	GCN	CCE	256	2	–	Adam 10^{-3}	100 warm-up + $\leq 4 \times 30^{*}$
		GAT		256	2	4×64		
		MLP		256	1	–		
		HGT		256	1	4×64		
	V	GAT		256	1	4×64		200
		MLP		256	2	–		

SGD: stochastic gradient descent.
For the mapping studies, layer counts exclude the linear classification head.
† Very small systems (fewer than 100 entities).
‡ GAT-DC: one layer for more than 500 files and more than 10 modules, two otherwise.
∗ Paper III epochs: warm-up plus at most the stated rounds of 30 epochs each, with pseudo-labels requiring confidence > 0.95.

5.3.1 Controlled-Comparison Logic

The studies examine selected choices within the design space through controlled comparisons. Some choices are coupled, since adding folder or member nodes also introduces relations and changes the paths available for message passing. A full factorial evaluation of all combinations would be computationally prohibitive. Each comparison therefore varies one or two choices while keeping the remaining conditions fixed where possible, allowing differences in outcome to be related to the choices under investigation.

The mapping studies compare graph-based encoders with an MLP using the same input features and training procedure. These comparisons are consistent with a contribution of dependency-graph information, but they change the predictor together with the available structure. Paper IV compares a dependency-

only graph with the same graph augmented by folder nodes while keeping the learning setting fixed, testing the contribution of organizational structure in the absence of lexical features. Paper V extends this logic by varying feature initialization and, with HGT, graph structure across file-only, folder-augmented, member-augmented, and combined graph variants. It also enables and disables dependency neighbors within each predictor while holding the file representations fixed. This makes it possible to examine the effects of input features and structural additions separately and, where combinations are included, their interaction.

5.4 Benchmark Systems

The studies evaluate open-source systems from published architecture recovery and C2A mapping benchmarks. Reusing expert-validated reference decompositions supports comparison with prior work and avoids introducing new architectural judgments, which require substantial domain knowledge and manual effort [20]. Table 5.3 lists the fourteen systems used across the thesis; each study evaluates the subset appropriate to its setting.

Table 5.3: Properties and versions of the benchmark systems. $\mathcal{M}$: number of modules in the ground-truth architecture; |SD|: static dependencies between files.

	System	Version	$\mathcal{M}$	|Files|	|SD|	Domain
Java	ArchStudio4	4	57	583	5 418	Architecture modeling
	ArgoUML	0.24	14	766	9 491	UML modeling
	Commons-Imaging	v1.0a2	21	329	4 561	Image I/O library
	Hadoop	0.19.0	67	591	10 510	Distributed processing
	JabRef	v3.7	6	1 013	16 394	Reference management
	Lucene	r1075001	7	1 054	18 595	Search engine
	OODT	0.2	216	876	8 811	Data management
	SweetHome-3D	r002382	9	167	3 912	Interior design
	TeamMates	v5.11	15	778	17 818	Peer feedback system
C	Bash	4.2	14	292	1 222	Shell
	HDF	0e196f	9	153	1 239	Driver subsystem
	Libxml	2.4.22	17	82	986	XML parser
C++	Chromium	svn-171054	69	18 343	284 435	Web browser
	HDC	46ff87	11	207	1 117	Camera interface

HDF: Hardware Driver Foundation; HDC: Distributed Camera.
Versions and counts refer to the benchmark instances used in the cross-study evaluation. They may differ from those reported in individual papers.

ArgoUML, Commons-Imaging, JabRef, Lucene, SweetHome-3D, and TeamMates are drawn from the benchmark assembled by Olsson et al. [64] and distributed through the s4rdm3x repository [63]. The mappings for JabRef and

TeamMates were produced by system experts during the SAEroCon workshops in 2016 and 2017. The ArgoUML and Lucene mappings were studied by Brunet et al. [6] and Lenhard et al. [48], respectively, whereas the Commons-Imaging mapping was contributed as an s4rdm3x test case.

For ArchStudio4, Hadoop, Bash, OODT, and Chromium, the thesis uses the reference decompositions curated by Garcia et al. [20, 50]. These decompositions were developed through a certification process that recovered an initial view from documentation and source code and refined it with system experts. The HDF, HDC, and Libxml mappings were developed with industry partners and are included in the SARIF benchmark studied by Zhang et al. [87].

5.5 Evaluation Protocol

Every controlled comparison in the thesis follows the same evaluation protocol. Model initialization and training, and, in the mapping studies, selection of the labeled set, involve randomness, so a single run cannot distinguish a systematic difference between configurations from ordinary run-to-run variation. Each stochastic method–configuration pair is therefore run 100 times. Results are summarized by the median across runs rather than the mean, because the median is robust to occasional outlying runs.

5.5.1 Labeled Training Sets

In the mapping studies, each labeled set begins with a uniform random sample of 5% of a system's files, bounded between 30 and 200 files. One file is then added from each module not represented in that sample. The floor and cap therefore apply to the random sample before these module-coverage additions, and the final labeled set can exceed the bounded sample size. Papers III and V use the same labeled sets, which self-training expands only in Paper III.

The 30-file floor applies to Commons-Imaging, SweetHome-3D, Bash, HDF, and HDC, corresponding to approximately 9–20% of their files before module-coverage additions. The 200-file cap applies only to Chromium, corresponding to approximately 1% of its files. For ArgoUML, JabRef, Lucene, and TeamMates, the random sample follows the nominal 5% rule. The experiments therefore use a bounded labeling budget rather than a constant labeled proportion: smaller systems receive a larger labeled share, while Chromium receives a smaller share. Comparisons within each system use the same labeled sets, but cross-system performance differences must be interpreted in light of these different labeled proportions.

The mapping studies therefore use ten of the fourteen systems. ArchStudio4, Hadoop, and OODT are excluded because representing every module would require labeling approximately 10–25% of their files, and Libxml is excluded because the 30-file floor alone would label approximately 37% of its files.

5.5.2 Statistical Analysis

To assess the statistical significance of observed differences between methods or configurations, nonparametric tests are used because metric values across repeated runs are not necessarily normally distributed. The recovery studies compare independent distributions of 100 runs per configuration and use the Mann–Whitney U test [17]. In the mapping studies, the compared configurations use the same labeled set in each run, producing paired observations. These comparisons use the Wilcoxon signed-rank test [85]. Where a recovered partition is compared with the ground-truth architecture, the 100 runs of a method are tested against the single ground-truth value of the normalized TurboMQ using the one-sample Wilcoxon signed-rank test.

Statistical tests use a significance threshold of $\alpha = 0.05$. For the mapping comparisons, the resulting p-values are adjusted using Holm's procedure across the ten systems, separately for each metric and each pair of compared configurations.

A significant result indicates that the observed difference would be unlikely if the two methods or configurations produced equivalent results, but it does not indicate the magnitude of the difference. With 100 runs per configuration, even a small difference can reach statistical significance. Differences are therefore also quantified using Cliff's delta, with effect sizes classified by magnitude as negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), or large ($|\delta| \geq 0.474$) [11, 75]. Between two configurations, delta is computed on the two sets of 100 scores as independent groups, and the significance test is paired. Against the ground-truth value, which is a constant, delta is the proportion of runs above that value minus the proportion below it. A difference that is significant but negligible is reported but not interpreted as an improvement.

Results are reported per system rather than averaged across systems, since averaging can conceal effects that differ across systems with different graph properties. The definitions and interpretation of the recovery and mapping metrics are given in Section 2.4.

5.6 Cross-Study Evaluation Design

The five studies were conducted on separately scoped benchmarks and, in part, with different input features and pipelines, so their reported results cannot be compared directly. For this thesis, four evaluations were therefore conducted in addition to the published work, followed by a correlation analysis with system properties. Their results are reported together with those of the studies in Chapter 6. Unless stated otherwise, the models, input features, training settings, and evaluation metrics are those described in Section 5.3.

The first two evaluations revisit recovery methods from Papers I, II, and IV on all systems in Table 5.3. Both use Ward-linkage agglomerative clustering of the learned file embeddings, with the number of clusters set to the number of modules in the ground-truth architecture. Each method is run 100 times per sys-

tem. Holding the clustering procedure fixed removes differences in clustering algorithms and cluster-count selection from the comparison.

The first evaluation compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using the GAT encoder. Paper I is extended to use the input features of Paper II while retaining its own training pipeline. The comparison therefore uses shared input features, although differences in training settings remain.

The second evaluation compares the attributed dependency graph of Paper II, using GCN and GAT, with the structure-only file-level and folder-augmented representations of Paper IV. It also compares the dependency cohesion of the recovered partitions with that of the ground-truth architectures.

The remaining two evaluations concern C2A mapping on the ten-system mapping benchmark. Within each comparison, the underlying file-level dependency graph, input features, 100 labeled sets per system, and training procedure are held fixed.

The third evaluation adds HGT to the comparison of Paper III to examine its performance with typed dependencies. It uses that study's file-level graph and input features, with two message-passing layers and messages aggregated from source to target, matching the depth and direction of its main configuration.

The fourth evaluation examines whether the contribution of dependency types varies with the input representation. It compares typed GAT, with one channel per relation type, against merged GAT, which collapses all types into a single relation, using the four input representations from Paper V: Word2Vec, Node2Vec, zero-shot UniXcoder, and fine-tuned UniXcoder. Both variants aggregate messages from source to target. None of these representations includes folder-path features, so this evaluation examines dependency types without organizational information in the node features.

5.6.1 Cross-Study Correlation with System Properties

This analysis assesses whether recovery performance varies systematically with system properties. Each system contributes one observation per method and configuration: its median metric value across the 100 runs, and these values are correlated with the system properties using Kendall's τ, a nonparametric rank correlation that accommodates tied ranks [41]. The sample size is therefore the number of systems, fourteen, and not the number of runs.

The properties are homophily $\mathcal{H}$, the proportion of static dependencies whose source and target files belong to the same ground-truth module [89], and average module size $\langle \mathcal{M} \rangle$, the mean number of files per ground-truth module, which characterizes the granularity of the target decomposition. Table 5.4 reports both properties and the normalized TurboMQ of the ground-truth decomposition.

Table 5.4: Properties of the benchmark systems, with the normalized TurboMQ of the ground-truth decomposition computed on the dependency-count-weighted graph. Shaded cells mark the highest value in each column.

System	Normalized TurboMQ TMQ_{norm}	Homophily $\mathcal{H}$	Avg. module size $\langle \mathcal{M} \rangle$
ArchStudio4	40.77	0.67	10.23
ArgoUML	59.27	0.55	54.71
Commons-Imaging	50.93	0.61	15.67
Hadoop	18.58	0.34	8.82
JabRef	31.78	0.56	168.83
Lucene	73.90	0.68	150.57
OODT	16.92	0.27	4.06
SweetHome-3D	13.60	0.36	18.56
TeamMates	9.17	0.34	51.87
Bash	45.29	0.58	20.86
Chromium	53.76	0.58	265.84
HDC	77.82	0.63	18.82
HDF	48.02	0.35	17.00
Libxml	35.08	0.21	4.82

Papers II and IV reported TMQ_{norm} without aggregating the multiplicity of member-level dependencies into a single file-pair edge weight, so the values here are not directly comparable.

Chapter 6

Research Findings and Synthesis

Contents

6.1 Results of the Cross-Study Evaluations 41
6.1.1 Recovery: Learning Objectives and Graph Inputs 41
6.1.2 Mapping: Predictors and Dependency Types 44
6.2 Synthesis of Research Findings 46
6.2.1 Objective 1: Representation Learning for Recovery and Mapping 46
6.2.2 Objective 2: Representations of Implementation Signals 47
6.2.3 Objective 3: Implementation Signals and Graph Structure 49

This chapter presents and synthesizes the findings against the three research objectives. It first reports the cross-study evaluation results, then combines them with the findings of the five studies to answer the research questions.

6.1 Results of the Cross-Study Evaluations

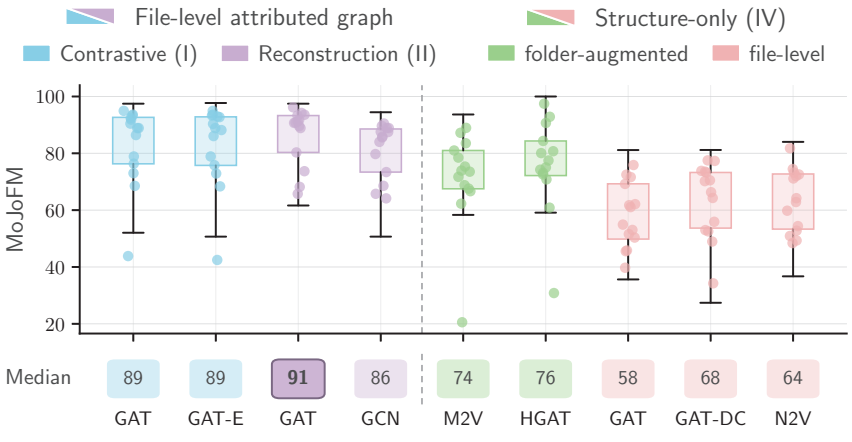
This section reports four cross-study evaluations in architecture recovery and C2A mapping. Boxplots pool 100 runs per method and system across all systems. They show the pooled distribution of outcomes without separating within-system from between-system variation. System-wise tests complement these plots.

6.1.1 Recovery: Learning Objectives and Graph Inputs

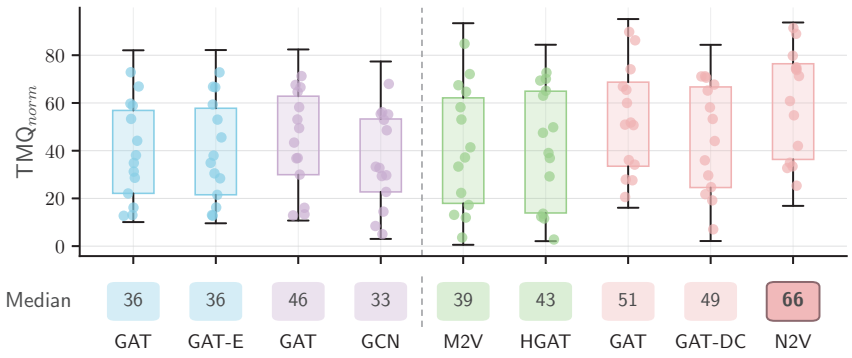
The two recovery evaluations cover all fourteen benchmark systems. The first compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using GAT and common node features. It also includes GAT-E, the edge-attribute variant of Paper I. The approaches retain their respective training settings, so the comparison does not isolate the learning objective alone.

The second compares the attributed-graph GCN and GAT of Paper II with the structure-only methods of Paper IV. These comprise GAT, GAT-DC, and Node2Vec on the file-level dependency graph, and MetaPath2Vec and HGAT on the folder-augmented graph.

MoJoFM, ARI, and $C2C_{cvg}$ measure agreement with the reference decomposition, while TMQ_{norm} measures dependency cohesion independently of that reference. Figure 6.1 presents the results using MoJoFM and TMQ_{norm}. The comparisons use a common clustering procedure, as described in Section 5.6.



(a) MoJoFM.



(b) Normalized TurboMQ

Figure 6.1: Recovery results across studies: agreement with the reference decomposition (a) and dependency cohesion (b). The attributed-graph methods are the contrastive (Paper I) and reconstruction (Paper II) variants; the structure-only methods (Paper IV) use the folder-augmented or the file-level graph. Boxplots pool all runs per system; dots mark system medians, and the boxed value indicates the median of these medians.

Agreement with the reference decomposition. In Figure 6.1(a), the contrastive and reconstruction variants have similar median MoJoFM values, but the contrastive variants show a wider spread toward lower scores. System-wise comparisons significantly favor the reconstruction-based GAT over the contrastive GAT on both MoJoFM and ARI for nine of the fourteen systems. Effects are large for ArchStudio4, Bash, Chromium, Libxml, and OODT, and small to medium for Commons-Imaging, HDF, SweetHome-3D, and TeamMates. Differences are not significant for ArgoUML, HDC, JabRef, or Lucene, while contrastive learning performs better only on Hadoop.

Adding edge attributes within the contrastive setup provides no consistent advantage. Mann–Whitney U tests find a significant difference between GAT and GAT-E only for MoJoFM on OODT, with a small effect favoring GAT.

Comparing graph inputs, attributed GCN and GAT achieve a higher median MoJoFM than all structure-only methods. ARI and $C2C_{cvg}$ give broadly similar rankings to MoJoFM when methods are ordered by their median scores across systems. The exceptions are that GCN ranks above the contrastive GAT variants under ARI, while GAT-DC and Node2Vec exchange places under $C2C_{cvg}$.

Dependency cohesion. Figure 6.1(b) shows a different ordering from reference agreement. The file-level structure-only methods achieve higher median TMQ_{norm} than the attributed methods, with Node2Vec attaining the highest value. Among the attributed methods, reconstruction-based GAT scores above the contrastive variants, while GCN has the lowest median of all methods. Reconstruction-based GAT also scores above the folder-augmented HGAT and MetaPath2Vec.

Because TMQ_{norm} varies widely across systems, each method is also compared with the reference architecture of the same system. Table 6.1 reports how many systems have significantly lower or higher scores, using one-sample Wilcoxon signed-rank tests. GCN falls below the reference on eleven of fourteen systems. The contrastive and folder-augmented methods fall below it on seven or eight systems, whereas the remaining methods do so on five or fewer. Node2Vec exceeds the reference on thirteen systems and falls below it only on Libxml.

Together, these comparisons show that agreement with the reference decomposition and dependency cohesion favor different methods. The attributed methods achieve higher reference agreement, while the file-level structure-only methods produce more dependency-cohesive partitions.

The common clustering procedure changes Node2Vec's results relative to its original k-means configuration. Using Ward linkage lowers the median ARI across systems from 0.293 to 0.266 and MoJoFM from 68.9 to 63.5, while increasing normalized TurboMQ from 63.9 to 66. Changes in reference agreement vary in direction across systems, whereas normalized TurboMQ increases on thirteen of the fourteen systems.

Table 6.1: Number of the fourteen systems on which a method's TMQ_{norm} is significantly below or above that of the ground truth (one-sample Wilcoxon signed-rank test, $p < 0.05$); on the remaining systems, the difference is not significant.

	Attributed graph (Papers I & II)				**Structure-only (Paper IV)**				
	Contrastive		Reconstruction		Folder-augmented		File-level		
	GAT	GAT-E	GAT	GCN	M2V	HGAT	GAT	GAT-DC	N2V
Below	8	8	5	11	8	7	2	5	1
Above	4	4	7	3	6	5	11	9	13

6.1.2 Mapping: Predictors and Dependency Types

The two mapping evaluations cover the ten-system mapping benchmark and use macro- and micro-F1. The first compares HGT with the MLP, GCN, and GAT of Paper III using the same file features and labeled sets. The graph-based predictors use a file-level graph with separate dependency relations.

The second compares typed and merged GAT using the four file representations from Paper V. It examines whether the benefit of preserving dependency types extends beyond the combined lexical and folder-path features used in Paper III.

Predictor comparison. Figure 6.2 shows that GAT has the highest median macro- and micro-F1. HGT's median micro-F1 lies between GCN and GAT, but its median macro-F1 is the lowest of all four predictors.

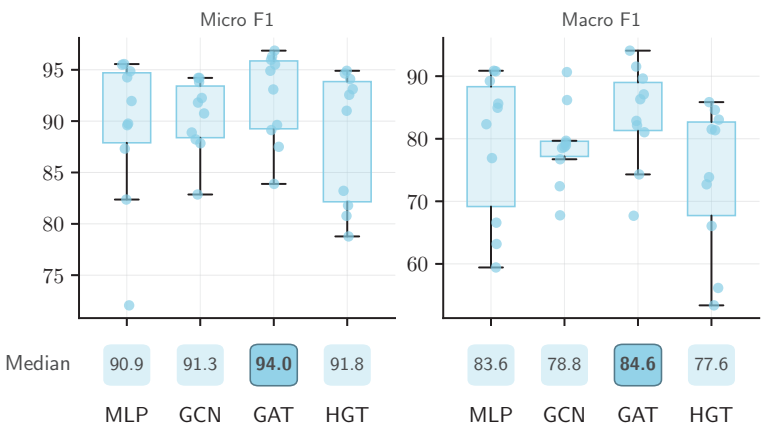


Figure 6.2: Macro- and micro-F1 for MLP, GCN, GAT, and HGT on the same input features and labeled-set splits. Boxplots pool all runs per system; dots mark system medians, and the boxed value indicates the median of these medians.

System-wise comparisons significantly favor GAT over HGT on nine of ten systems for macro-F1 and eight for micro-F1, with mostly large effects. HGT significantly outperforms GAT only on Chromium's micro-F1, by 1.9 points with a large effect.

In the system-wise tests, HGT does not generally outperform MLP and GCN. Compared with MLP, it scores significantly lower on seven systems for each metric, with small-to-large effects, and higher on Chromium and JabRef for both. Compared with GCN, it scores significantly lower on six systems for macro-F1 and five for micro-F1, and higher on two and four systems, respectively.

Effects of dependency type modeling. Table 6.2 compares typed and merged GAT using Word2Vec, Node2Vec, and zero-shot and fine-tuned UniXcoder representations. Both variants use the GAT architecture of Paper III with source-to-target aggregation. None of the input representations includes folder information.

Preserving dependency types yields positive median F1 differences across systems for all four representations. The largest median gain is for Word2Vec in macro-F1, at 5.8 points, and for fine-tuned UniXcoder in micro-F1, at 2.8 points. System-wise tests show significant gains on seven of ten systems for Word2Vec under both metrics, and on seven systems for fine-tuned UniXcoder in macro-F1 and nine in micro-F1.

The benefit is not uniform. Every representation also shows significant losses on some systems. Bash is a recurring exception, with significant losses for Word2Vec and both UniXcoder representations under both metrics. Zero-shot UniXcoder shows particularly mixed effects, with large gains on SweetHome-3D and HDF and large losses on Bash and Chromium.

Table 6.2: Typed minus merged GAT across the ten systems, by input vector and metric. Δ: median F1 difference in points. +ve: number of systems with a positive difference. ↑/↓: number of systems with a significant gain/loss (paired Wilcoxon signed-rank test, Holm-corrected across the ten systems for each vector and metric, $\alpha = 0.05$). N/S/M/L: number of systems with negligible, small, medium, or large Cliff's δ between the typed and merged scores.

	Macro-F1					Micro-F1				
Vector	Δ	+ve	↑	↓	N/S/M/L	Δ	+ve	↑	↓	N/S/M/L
N2V	+0.6	6	5	1	5/4/0/1	+0.8	7	7	3	4/4/0/2
W2V	+5.8	8	7	1	2/0/1/7	+1.4	9	7	1	1/4/1/4
UXC_{ZS}	+0.1	5	4	4	2/3/1/4	+0.1	5	5	3	2/2/2/4
UXC_{FT}	+2.6	7	7	3	1/3/1/5	+2.8	9	9	1	2/1/2/5

6.2 Synthesis of Research Findings

This section combines the findings of the five studies with the cross-study results in Section 6.1 to answer the research questions. The synthesis follows the three research objectives, examining the use of representation learning for recovery and mapping, the information provided by representations of individual implementation signals, and the contribution of additional graph structure.

6.2.1 Objective 1: Representation Learning for Recovery and Mapping

Objective 1 examines how graph-based representation learning supports architecture recovery and C2A mapping. The findings address the combination of implementation signals (RQ1.1), the treatment of dependency types (RQ1.2), and the effects of learning objective, encoder architecture, and depth (RQ1.3).

RQ1.1: Joint representation of implementation signals. Papers I–III combine dependencies, code identifiers, and folder organization to learn representations for recovery and mapping. These methods improve on their respective baselines in reference agreement for recovery and F1 for mapping. The results support the combined representations, but do not establish how much each signal contributes.

In Paper III, an MLP using the same file features also outperforms the Naive Bayes baseline on every system under both F1 metrics. Improvement over that baseline therefore does not, by itself, demonstrate a benefit from dependency-based message passing.

RQ1.2: Modeling dependency types. Papers I–III represent dependency types as separate relations, each processed by its own encoder channel, and combine the outputs by summation. Paper III compares this design with merging all dependencies into a single relation. With lexical and folder-token features, separate relations significantly improve both GCN and GAT across all evaluated systems. However, the separate channels also add parameters, so the comparison does not isolate dependency typing from model capacity. Papers I and II do not test a merged alternative.

The cross-study mapping comparison extends this evaluation to the folder-free representations of Paper V. Separate relations generally benefit GAT, but the benefit is less consistent than in Paper III and varies across representations and systems, with significant losses on some systems (Section 6.1.2). HGT does not match GAT on the typed dependency graph, showing that the use of dependency types alone does not determine predictor performance.

Recovery provides less consistent support for separate relations. In Paper IV, preliminary experiments on file-level dependency graphs showed markedly lower performance with separate relations, so the reported dependency-only

comparisons use merged dependencies. With folder nodes and containment relations included, preserving dependency types gives modest benefits that vary across systems and evaluation measures.

Overall, separate dependency relations benefit mapping more consistently than recovery in the evaluated settings. Their effects vary with the input representations and encoder, but differences between the studies prevent these factors from being isolated.

RQ1.3: Learning objective, encoder architecture, and depth. The studies examine how these three choices affect recovery and mapping.

- *Learning objective:* The cross-study evaluation in Section 6.1.1 compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using common node features and a GAT encoder. Reconstruction generally achieves better agreement with the reference decomposition, although the advantage is not universal. Because the approaches retain different training settings, the comparison supports the reconstruction-based approach without isolating the learning objective alone.
- *Encoder architecture:* GAT generally outperforms GCN across recovery and mapping (Papers II and III), and HGT in the cross-study mapping comparison. GCN uses degree-normalized aggregation, whereas GAT learns weights for neighboring representations. However, the models also differ in parameterization, so these results cannot be attributed to attention alone. In mapping, GAT usually outperforms the MLP, while GCN frequently underperforms it. Access to dependencies therefore does not guarantee better predictions.
- *Encoder depth:* Paper III finds no consistent benefit from increasing message-passing depth. A second layer improves performance on some systems and reduces it on others.

Summary of Objective 1

Combining dependency, lexical, and folder signals improves on the evaluated baselines in recovery and mapping, although the contribution of each signal is not isolated (RQ1.1). Preserving dependency types benefits mapping more consistently than recovery, with effects varying across inputs and encoders (RQ1.2). The reconstruction-based approach generally achieves better reference agreement than the contrastive approach, but their training settings also differ. GAT generally outperforms GCN across both tasks and HGT in the cross-study mapping comparison, while increasing encoder depth has no consistent benefit (RQ1.3).

6.2.2 Objective 2: Representations of Implementation Signals

Objective 2 (Section 3.2.2) examines the architectural information provided by representations of individual implementation signals. The findings address re-

covery from dependency structure alone (RQ2.1), dependency-guided adaptation of contextual code representations (RQ2.2), and the comparison of structural, lexical, and contextual file representations (RQ2.3).

RQ2.1: Recovery from dependency structure. Paper IV shows that dependency structure alone supports recovery. Node2Vec and GAT-DC outperform the classical recovery baselines, while plain GAT is generally weaker than both.

Structural reach helps explain this difference. Node2Vec and GAT-DC achieve better reference agreement than the two-layer GAT on most systems. Applying diffusion to the GAT input graph closes much of the gap to Node2Vec, suggesting that access to a wider dependency neighborhood contributes to recovery performance.

The evaluation criterion matters as well (Section 6.1.1). Node2Vec produces the most dependency-cohesive partitions, while GAT-DC improves reference agreement over plain GAT on most systems without a corresponding advantage in dependency cohesion.

RQ2.2: Dependency-guided adaptation of code representations. Paper V finds that dependency-guided fine-tuning of UniXcoder improves mapping performance over its zero-shot form on nearly all systems. Both representations are held fixed during classifier training and evaluated using the same MLP and labeled sets, without folder information or additional message passing.

Fine-tuning uses a contrastive objective with positive pairs drawn mainly from two-hop dependency neighborhoods. This choice follows the observation in Paper IV that structural context beyond direct neighbors can support recovery. With the classifier and evaluation conditions held fixed, the gains support adapting the pretrained representation to the target system using dependency information.

However, the comparison does not isolate the contribution of dependency-based pair selection, since it includes no fine-tuning alternative using pairs selected without dependency information.

RQ2.3: Comparing file representations. Paper V compares structural, lexical, and contextual file representations using the same graph-free MLP and labeled sets, without folder information or additional message passing. Representations that encode dependency information generally perform better. Node2Vec captures dependencies through random walks, while fine-tuned UniXcoder incorporates them through dependency-guided adaptation. Both generally outperform Word2Vec and zero-shot UniXcoder, although there are exceptions, including JabRef, where Word2Vec is among the strongest. Word2Vec and zero-shot UniXcoder show no consistent ordering, with each significantly outperforming the other on several systems.

Neither Node2Vec nor fine-tuned UniXcoder is uniformly best. Node2Vec performs significantly better on six systems under both F1 metrics, while fine-tuned

UniXcoder performs significantly better on three systems under both metrics and additionally on HDC under macro-F1.

Together with the improvement from zero-shot to fine-tuned UniXcoder (RQ2.2), these findings suggest that dependency information contributes to the stronger representations. However, the representations also differ in content and training procedure, so the contribution of dependency information alone cannot be isolated.

Summary of Objective 2

Dependency structure alone supports recovery, with performance depending on structural reach and how closely dependencies follow the reference modules. Reference agreement and dependency cohesion favor different methods (RQ2.1). Dependency-guided fine-tuning improves UniXcoder over its zero-shot form on nearly all systems, although the contribution of dependency-based pair selection is not isolated (RQ2.2). Node2Vec and fine-tuned UniXcoder generally outperform Word2Vec and zero-shot UniXcoder in mapping without folder information or additional message passing, but neither is uniformly best. Differences in content and training prevent the advantage from being attributed solely to dependency information (RQ2.3).

6.2.3 Objective 3: Interaction Between Implementation Signals and Graph Structure

Objective 3 (Section 3.2.3) examines how graph structure contributes beyond the initial file representations. The findings address dependency-neighbor aggregation and its direction (RQ3.1), folder organization (RQ3.2), and the additional detail provided by member nodes and edge attributes (RQ3.3).

RQ3.1: Dependency neighbors and aggregation direction. Paper V examines the contribution of dependency neighbors by comparing HGT, GAT, and a neighbor MLP with and without neighbor aggregation, holding file representations fixed. The neighbor MLP appends the mean of neighboring files' vectors to each file's own vector. These comparisons assess neighbor access within each predictor, avoiding the change in predictor architecture involved in comparing a GNN with a graph-free MLP.

Dependency neighbors mostly improve mapping beyond the information already present in the file representations, but the gains vary across predictors and inputs. For GAT and the neighbor MLP, gains are larger for Word2Vec and zero-shot UniXcoder, which do not encode dependencies, than for Node2Vec. However, fine-tuned UniXcoder benefits more than Node2Vec. GAT is generally the strongest predictor, while HGT gains the least from neighbor access. With GAT, fine-tuned UniXcoder scores higher than Node2Vec in micro-F1 on seven of the ten systems with either aggregation direction, up from three without aggregation (RQ2.3), whereas in macro-F1 the count rises only from four to five.

Aggregation direction also affects performance. Source-to-target aggregation updates a file using information from files that depend on it, while undirected aggregation permits information to flow in both directions. In Paper V, source-to-target aggregation lowered median macro-F1 with Node2Vec, which already encodes dependency context, whereas undirected aggregation improved it. The preferred direction varied across systems rather than across predictors or input representations. Systems that combine low homophily with high average degree favored source-to-target aggregation, while several sparser systems favored undirected aggregation. However, making the graph undirected also adds reverse edges, so the comparison does not separate direction from neighborhood size, and the ten systems do not separate connectivity from homophily.

Paper III examines aggregation direction for GAT and GCN and finds source-to-target aggregation generally preferable for GAT. This preference agrees with Paper V on some systems but differs on others. The studies also differ in dependency typing, node features, depth, and training, so neither the benefit of aggregation nor the preferred direction can be assumed to transfer across settings.

These comparisons isolate neighbor access in mapping. The dependency-only recovery methods have no corresponding condition without dependency context. Their results instead show how recovery varies with structural reach and how closely dependencies follow reference-module boundaries (RQ2.1).

RQ3.2: Folder organization. The studies represent folder organization either through directory tokens in file features (Papers I–III) or through folder nodes and containment relations (Papers IV and V). In recovery, methods that include organizational information achieve better agreement with the reference decomposition, particularly where dependencies provide weak evidence of module boundaries. Dependency-only methods instead produce more dependency-cohesive partitions (Section 6.1.1).

In mapping, Paper V finds that adding folder nodes produces the largest and most consistent gains across the evaluated additions to the graph, with Word2Vec benefiting most. Across these configurations, folder nodes use the same folder-token features. With fine-tuned UniXcoder, the folder-aware model approaches the mapping performance of a rule based on directory hierarchy and the initial labeled set on most systems, exceeding it only on Lucene.

The overall gains do not establish a consistent benefit for files that the directory-based rule assigns incorrectly. On these files, the folder-aware model is more accurate than the model without folder nodes on some systems, similar on others, and less accurate on others, and on most of the systems where the rule makes errors it still assigns most of them incorrectly. The analysis does not isolate files whose module differs from the majority module of their directory. Folder location is therefore informative for the reference mappings, while whether the model handles such files reliably remains untested. These mapping findings apply to HGT with source-to-target dependency aggregation and hold with one or two layers.

RQ3.3: Member nodes and edge attributes. Neither member-level nodes nor edge-level interaction attributes consistently improve performance in the evaluated settings.

In Paper V, adding member nodes to the file-only graph lowers the median F1 for all three representations evaluated with HGT. When folder nodes are also present, the effect of adding member nodes varies with the representation. These additions change both the graph structure and how features enter the model, so their effects cannot be attributed to structure alone.

Paper I reports small gains from edge attributes on some systems. However, the cross-study evaluation finds no significant improvement over the same model without these attributes (Section 6.1.1). Paper I attributes the limited benefit on TeamMates to edge attributes amplifying noisy cross-module interactions, suggesting that additional interaction detail may be less useful where dependencies poorly reflect module boundaries.

Summary of Objective 3

Dependency-neighbor aggregation improves mapping beyond the initial file representations, most reliably for GAT and the neighbor MLP. The magnitude of improvement depends on both the predictor and the input representation, with fine-tuned UniXcoder gaining more than Node2Vec, while the preferred aggregation direction varies across systems (RQ3.1). Folder organization provides the strongest additional signal in the evaluated mapping configurations, but does not consistently help correct files misclassified by the directory-based rule. In recovery, higher agreement with the reference architecture can coincide with weaker dependency cohesion (RQ3.2). Adding finer-grained structure does not extend these benefits consistently. Member-level nodes give no consistent improvement in C2A mapping, and edge-level attributes give no significant gain in the architecture recovery (RQ3.3).

Chapter 7

Discussion

Contents

7.1 Interpretation of the Findings 53
7.1.1 Reference Alignment and Dependency Cohesion 53
7.1.2 System Properties and Recovery Performance 54
7.1.3 Structural Reach and Neighborhood Aggregation 55
7.2 Limitations 56

This chapter interprets the findings across the five studies and presents an exploratory analysis of how system properties relate to recovery performance. It examines differences between recovery-method rankings under reference agreement and dependency cohesion, the relationship between dependency homophily and recovery performance, and the role of structural reach in recovery and mapping. It concludes with the limitations of the work. Unless directly tested, the proposed explanations remain interpretations of the observed patterns.

7.1 Interpretation of the Findings

The findings raise three questions for interpretation: why evaluation criteria favor different methods, how system properties relate to recovery performance, and how the available dependency context affects recovery and mapping. The following subsections examine these questions across the studies.

7.1.1 Reference Alignment and Dependency Cohesion

In the cross-study recovery comparison (Section 6.1.1), attributed GCN and GAT show the strongest agreement with the reference decomposition, whereas file-level structure-only methods produce more dependency-cohesive partitions. Node2Vec has the highest median normalized TurboMQ. Several factors may explain these different rankings.

TurboMQ rewards dependency concentration within clusters, while file-level structure-only methods learn exclusively from the dependency graph. Although their objectives do not explicitly optimize cohesion or coupling, learning from dependency proximity may favor partitions that score well on this measure.

Attributed representations also include folder paths and identifiers, which reflect how developers organize and describe responsibilities. These signals may support agreement with the reference decomposition without increasing dependency cohesion.

The reference decomposition itself need not maximize dependency modularity. Utility or cross-cutting code may receive dependencies from several modules, so matching its reference assignment need not produce the most dependency-cohesive partition.

The compared methods differ in graph construction, input information, and learning configuration. These explanations are therefore plausible interpretations of the observed rankings, but the comparisons do not isolate their individual contributions.

7.1.2 System Properties and Recovery Performance

We examine associations between recovery performance and benchmark-system properties. For each method in the cross-study recovery evaluation, we correlate per-system median scores with dependency homophily and average module size using Kendall's τ across the fourteen systems (Figure 7.1). Given the small sample and multiple tests, these correlations are treated as exploratory associations rather than independent effects.

Dependency homophily shows the clearest association with performance, particularly for structure-only methods. The structure-only GAT, GAT-DC, and Node2Vec all show significant positive associations with homophily for MoJoFM and TMQ_{norm}, while only the GAT does so for ARI. The attributed GCN and GAT show weaker and less consistent associations. Lexical and file-location attributes may provide complementary evidence when dependencies cross module boundaries, but these comparisons do not isolate their contribution from other methodological differences.

The Node2Vec results are consistent with its reliance on dependency structure alone, so its sensitivity to homophily may be a property of the method and not only of the benchmark. Random walks define the contexts from which file representations are learned, without module labels to distinguish within-module from cross-module dependencies. When dependencies frequently cross module boundaries, files from different modules may therefore appear in similar walk contexts. The positive associations with MoJoFM and TMQ_{norm} are consistent with this interpretation but do not establish the mechanism. Because homophily is computed from the reference decomposition, it provides a retrospective explanation and cannot guide the choice of a representation in advance.

Associations involving TMQ_{norm} need a separate reading. Homophily describes dependency concentration within reference modules, whereas TMQ_{norm} rewards dependency concentration within recovered clusters and is normalized by the number of clusters, so its associations with homophily and module size are partly built into the metric. They describe the structural quality that TMQ_{norm} measures and do not independently support an effect on recovery accuracy.

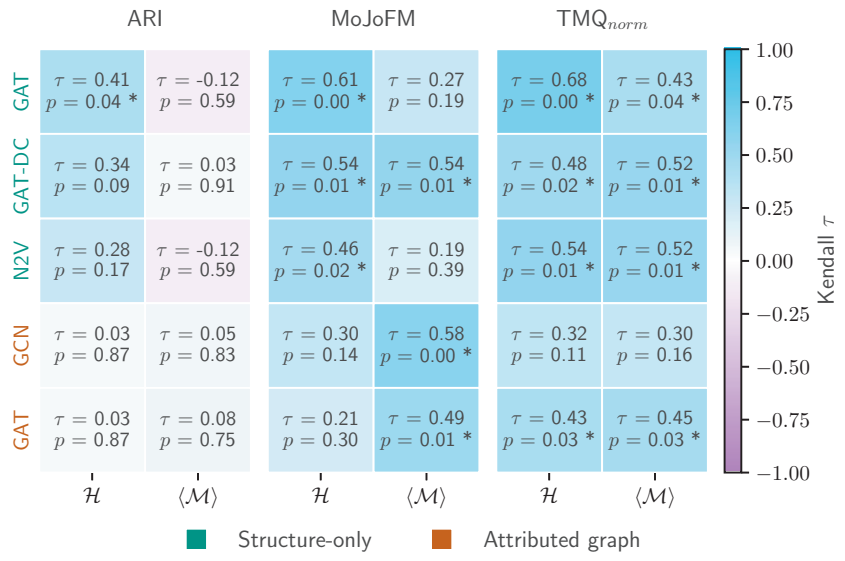


Figure 7.1: Kendall's τ correlation between recovery outcomes (ARI, MoJoFM, TMQ_{norm}) and two system properties, homophily $\mathcal{H}$ and average module size $\langle\mathcal{M}\rangle$, per method. GAT, GAT-DC, and N2V (structure-only) operate on the homogeneous dependency graph alone; GCN and GAT (attributed graph) operate on the multi-relational dependency graph with node features derived from file location and code identifiers.

Average module size is positively associated with MoJoFM for three of the five methods, but with ARI for none. This pattern is difficult to interpret independently of other system properties: OODT and Libxml have both the smallest modules and the lowest homophily, while larger modules tend to occur in larger systems. With fourteen systems, these associations cannot be separated, so no independent effect of module size is inferred.

7.1.3 Structural Reach and Neighborhood Aggregation

We use structural reach to describe the extent of the dependency neighborhood from which a file's representation can incorporate information through message passing, diffusion, or random walks. The studies vary these mechanisms rather than measuring reach directly.

In dependency-only recovery, Paper IV finds that GAT-DC and Node2Vec outperform plain GAT overall. GAT-DC makes diffusion-close nodes directly accessible to the encoder, while Node2Vec learns from contexts formed by random walks. These results suggest that the two-hop neighborhood available to plain GAT provides insufficient context on some systems.

However, GAT-DC also uses system-specific settings for depth and the number of retained diffusion neighbors. Paper IV examines sensitivity to the retained-neighbor count, but the comparison with plain GAT does not isolate the contribution of structural reach under matched settings.

Increasing depth offers another way to access more distant nodes, but it is not necessarily equivalent to diffusion or random walks. Additional layers repeatedly aggregate information over the same graph, diffusion changes which nodes are directly connected, and random walks define the contexts used to learn representations. Paper III finds mixed effects from adding layers in mapping, with neither connectivity nor homophily alone explaining when greater depth helps. These mechanisms were not compared under matched conditions across tasks.

Where dependencies cross module boundaries, additional layers may mix information from different modules. Repeated aggregation may also make node representations less distinguishable through over-smoothing [49]. Neither mechanism was measured directly in these studies, so both remain possible explanations for the mixed effects of depth.

In mapping, Paper V shows that encoding dependency context in the input representation does not remove the potential benefit of neighbor aggregation during prediction. With file vectors held fixed, adding neighborhood aggregation produces different gains across predictors and directions, including when the input vectors already encode dependencies through Node2Vec. The usefulness of additional context therefore depends on how it is combined with the existing representation.

Aggregation direction also varies in usefulness across settings. Papers III and V favor different directions on several systems, but also differ in dependency typing, features, depth, and training. These differences were not individually tested as explanations. Moreover, undirected aggregation adds reverse edges, so its comparison with source-to-target aggregation does not separate direction from neighborhood size.

Taken together, the studies suggest that both the available dependency context and the mechanism used to incorporate it matter. Wider reach can support recovery and mapping, but its benefit depends on the initial representation, the predictor, and how closely dependencies follow module boundaries.

7.2 Limitations

The conclusions are limited by the systems, architectural representations, and methodological choices examined. Some limitations apply across all five studies, while others concern particular graph constructions, model configurations, or evaluation protocols.

Benchmark scope and external validity. The studies evaluate overlapping subsets of open-source Java, C, and C++ systems with published reference decompositions. The findings may not generalize to other programming languages,

application domains, build environments, or architectural styles. Although the methods may be applied without a reference decomposition, agreement with an intended architecture cannot be assessed without a comparable reference. Chromium is the only large-scale system evaluated, so evidence of scalability is limited to a single system. Each system is evaluated at a particular version; the studies therefore do not establish robustness to refactoring or changes in dependency structure across releases.

The exploratory analysis across fourteen systems does not establish whether dependency homophily affects recovery independently of system size, module granularity, or other system properties.

Dependency graph. Static dependencies are extracted automatically with the Depends [62] tool. The resulting graphs may omit runtime dependencies or dependencies the tool cannot resolve, and may include edges not realized at runtime. Recovery and mapping results therefore depend on the extracted graph's accuracy and coverage. Within this thesis, this limitation may affect dependency-derived signals, including dependency-guided fine-tuning of contextual embeddings. Folder and containment relations, lexical embeddings, and zero-shot contextual embeddings do not depend on this extraction. The studies neither independently validate extraction accuracy nor assess sensitivity to extraction errors.

Reference architectures and architectural representation. The reference decompositions were produced by system experts, industry partners, or researchers reconstructing architecture from documentation, and only one reference is available per system. They are not uniquely determined ground truths: reasonable disagreement may arise when module boundaries are unclear, files implement several concerns, or the intended architecture differs from the implementation. Source revisions and reference mappings are not always aligned, and build configuration determines which files are analyzed, so assignments may be outdated or missing. In the mapping studies, labels are sampled from the same reference used for evaluation, so an incorrect assignment can misdirect training and penalize a plausible prediction.

Architecture granularity is a modeling choice: the same system can be represented by a few large modules or by more, smaller submodules. JabRef, for example, has six modules of very uneven size, so one of the larger ones could plausibly be split into three, whereas the 15 modules of TeamMates could be merged into six higher-level ones according to their name prefixes. Merging modules keeps existing internal dependencies within modules and may make inter-module dependencies internal as well, so homophily can only increase or stay unchanged; splitting has the opposite effect. Even with the same dependency graph, the chosen granularity can therefore change homophily, the reference partition's TurboMQ, and potentially the recovery and mapping scores.

More generally, both tasks represent architecture as a flat, exclusive partition of files into modules. Files implementing multiple concerns must be assigned to a

single module; overlapping responsibilities, cross-cutting infrastructure, and hierarchical decompositions are not evaluated. The reported scores therefore measure agreement with one file-level architectural view, not whether the recovered or predicted architecture fully represents the system.

Mapping protocol. The random labeled sample comprises 5% of files, bounded between 30 and 200, with five systems at the 30-file minimum, so label count and proportion vary with system size and were not varied independently. The mapping models are also transductive, since all system files are available during training.

Organizational signals and shared provenance. Folder organization is the strongest additional signal identified under Objective 3, but it introduces a validity concern. Folder hierarchies and reference decompositions may reflect shared design decisions: developers may organize folders around intended module boundaries, and researchers may use folder structure when constructing reference decompositions. A directory-based baseline in the mapping study agrees closely with the reference mappings, but the extent of this overlap was not otherwise quantified. Folder paths also inform file-node features in Papers I–III and initialize folder nodes in Paper V. The gains from folder information may therefore partly reflect shared provenance between inputs and reference labels rather than independent architectural evidence. Systems with reorganized, flattened, or architecture-inconsistent directory structures were not evaluated.

Representation, objective, and comparison choices. The studies use graph abstractions that encode different notions of structural relevance. Several models use binary file-level adjacency, which omits dependency frequency. Paper I includes member-level interaction counts and types as edge attributes, whereas Paper IV uses PPR weights to represent topology-derived proximity. These represent different notions of edge strength and were not compared across encoders and tasks. Aggregation direction was varied only in mapping, so its effect on recovery remains untested. Member-level information was evaluated only as an aggregate node per file and with one encoder, HGT, so the results do not show that it is uninformative under other designs or encoders.

Recovery objectives optimize proxies rather than agreement with the intended architecture, and results also depend on the clustering method and, in the recovery studies, on the number of clusters, which was searched over a range. The cross-study evaluations fix the cluster count to the reference count, which improves comparability but supplies information generally unavailable in practice.

Between-method comparisons involve differences in graph construction, inputs, training, depth, and parameter count, which limit attribution of performance differences to individual choices. Hyperparameter selection on the evaluation systems using reference-based metrics may also favor more extensively tuned methods.

Stochasticity and reproducibility. Results are summarized by medians over 100 runs per configuration in all five studies. Input representations were computed once and reused, except for Node2Vec in recovery, which was recomputed for each run. Reported variation therefore excludes the stochasticity of the fixed representations. With 100 runs, even small differences can reach statistical significance without being practically meaningful. Effect sizes are therefore reported alongside p-values, and interpretation should consider effect magnitude rather than significance alone.

In the mapping studies, each run pairs a labeled set with a training seed, so their contributions to variation are not separated in the main evaluation. When Paper III varied the training seed with the labeled set held fixed, the ranking of a few configurations changed across seeds, whereas some advantages were robust. These results indicate seed sensitivity, but the limited comparisons do not characterize it fully. Close rankings should therefore be interpreted cautiously.

Chapter 8

Conclusion and Future Work

Contents

8.1 Conclusion 61
8.2 Future Work 63

This chapter concludes the thesis by synthesizing findings on graph-based representation learning for architecture recovery and C2A mapping. It summarizes the conclusions across the three research objectives and outlines directions for further research arising from the findings and limitations of the work.

8.1 Conclusion

This thesis examined how graph-based representation learning can support software architecture recovery and C2A mapping, and what different implementation signals contribute to these tasks. Recovery was studied as an unsupervised task and mapping as a supervised task with limited labeled data. Across five studies, the work progressed from learning joint representations of static dependencies, folder organization, and source-code content to examining their individual contributions and their interactions with graph structure.

The first objective examined whether dependency, lexical, and organizational signals could be combined in learned representations for recovery and mapping, and how performance depended on the learning objective and model configuration. The joint representations supported both tasks, although the evidence differed between them. In recovery, the approaches of Papers I and II improved on or matched the evaluated baselines across most comparisons. In mapping, however, improvement over the Naive Bayes baseline was not specific to the GNN models. Both the GNN models of Paper III and an MLP using the same features outperformed that baseline. This improvement therefore does not by itself establish a benefit from message passing.

In the recovery comparison using a common GAT encoder and input features, the reconstruction-based approach generally produced better agreement with the reference decompositions than the contrastive approach. The approaches retained different training settings, so the comparison does not isolate the learning objective alone. Encoder choice also mattered: GAT generally outperformed

GCN in both tasks. Modeling dependency relations separately improved mapping with lexical and folder-token features (Paper III), although the encoder also had more parameters. With file vectors without folder-path features, the benefit was smaller and varied across representations and systems, as shown in the cross-study mapping evaluation. Separate relations gave no benefit in preliminary tests in dependency-only recovery (Paper IV). Relation-specific modeling was therefore useful under some input representations and tasks, rather than consistently beneficial.

The joint-signal comparisons did not separate the contributions of the individual signals. The second objective therefore examined what architectural information static dependencies and source-code content carry, and how representation choices make that information available. In recovery from static dependencies alone, the results suggest that the extent of dependency context encoded in a file's representation (structural reach) matters. Node2Vec extends this reach through random walks and GAT-DC through PPR diffusion. Both outperformed the classical baselines on most systems, whereas the two-layer GAT, limited to its two-hop neighborhood, was generally weaker. Agreement with the reference decomposition was lower where within-module and cross-module dependencies were similarly dense, whereas the dependency cohesion of the recovered partitions frequently matched or exceeded that of the reference. Dependency structure alone therefore supported recovery to a bounded extent: its usefulness depended on structural reach, on how well the reference modules were separated by dependencies, and on the evaluation criterion.

In mapping with an MLP, Word2Vec and zero-shot UniXcoder showed no consistent ordering, so the pretrained code transformer was not consistently better than a lexical representation. Dependency-guided fine-tuning improved UniXcoder on nearly all systems, supporting adaptation of the pretrained representation to the target system using dependency information. Node2Vec and fine-tuned UniXcoder generally outperformed the other two representations. Neither of the two was uniformly best: Node2Vec most often performed best, while fine-tuned UniXcoder was significantly stronger on some systems. Dependency information was therefore useful in mapping as in recovery, regardless of representation family, although differences in content and training prevent attributing the performance differences to dependency information alone.

The information carried by individual file representations did not establish what additional graph context could contribute. The third objective therefore examined how dependency-based message passing, folder organization, and member-level detail changed their usefulness. With undirected aggregation and fixed file representations, GAT and an MLP supplied with mean neighbor vectors improved on most systems for every representation, with no significant losses. Gains were largest for zero-shot UniXcoder and smallest for Node2Vec. Fine-tuned UniXcoder gained more than Node2Vec and had the highest median macro-F1 across systems for every predictor. The benefit of aggregation therefore depended on how dependency context was already encoded.

Aggregation direction also mattered. Source-to-target aggregation lowered median macro-F1 with Node2Vec, and the preferred direction depended on the system rather than the predictor or representation. Papers III and V agreed on some systems but differed on others, so neither the benefit nor the preferred direction can be assumed to transfer across setups. Undirected aggregation also added reverse edges, so direction and neighborhood size were not separated.

Folder organization supplied architectural evidence beyond static dependencies alone. Across the recovery studies, methods using organizational information agreed more closely with the reference decomposition, particularly in Paper IV where dependencies gave limited evidence of module boundaries, but produced less dependency-cohesive partitions than file-level dependency-only methods. Reference agreement and dependency cohesion therefore favored different configurations. In mapping with HGT and source-to-target dependency aggregation, folder nodes improved every file representation, but the model with fine-tuned UniXcoder did not consistently outperform the directory-based baseline, limiting the evidence for a benefit beyond directory organization itself (Paper V). Folder information and the reference decomposition may also overlap because directories are consulted during reference construction; the extent of this overlap was not quantified.

Member-level detail gave no consistent improvement in the evaluated configurations. With HGT and source-to-target dependency aggregation, member nodes lowered median mapping F1 for all three representations without folders and had representation-dependent effects with folders (Paper V). Since member nodes also changed how features entered the model, their effects cannot be attributed to the added structure alone. In the cross-study recovery evaluation, member-level edge attributes gave no significant improvement over the same model without these attributes. These findings therefore concern the evaluated encoders and representations, not the usefulness of member-level information in general.

Taken together, dependency information remained useful, but encoding it in file representations did not make message passing redundant, and folder organization supplied further information. The dependence on encoder choice observed under Objective 1 recurred as a dependence on predictor and aggregation direction, and across the studies no representation or configuration was uniformly preferable: the contribution of each signal depended on how it was represented and combined with other evidence.

8.2 Future Work

A first direction is to determine which aspects of graph context account for the observed performance differences. Undirected aggregation changed both propagation direction and neighborhood size, while member nodes changed graph structure and feature placement. Comparing incoming, outgoing, and combined neighborhoods at matched sizes could help distinguish direction from neigh-

borhood size, while member-level comparisons should preserve feature content. Parameter-matched comparisons of relation-specific and collapsed-relation models could distinguish relational detail from model capacity.

Paper I included interaction frequency and type together as member-level edge attributes. The cross-study recovery evaluation found no significant improvement from these attributes, but did not isolate the contribution of frequency. Whether frequency adds architectural information, and how it should enter representation learning through edge weights, attention terms, or relation encodings, remains open for both recovery and mapping.

A second direction concerns configuration selection and evaluation on unfamiliar and evolving systems. The benefit and preferred direction of neighbor aggregation varied across systems, but dependency homophily, used to interpret these differences, requires reference module assignments. Future work should examine whether properties observable in the dependency graph alone can guide configuration selection, with selection rules evaluated on systems not used to develop them. Folder-aware methods could also be assessed across systems that differ in how closely directories match the reference modules. Reporting performance separately for files whose module differs from that of their directory would help establish whether the methods provide useful predictions beyond directory organization.

The mapping evaluations made all system files available, leaving performance on subsequently introduced files untested. Future work could evaluate whether a model trained on mapped files from one version can assign new and changed files in later versions without retraining. The version history could also supply evolutionary signals, such as files that change together, which may carry architectural information that dependencies, folders, and code content do not. Practitioner studies could then assess whether the predictions support architecture maintenance.

Bibliography

[1] Nour Ali, Sean Baker, Ross O'Crowley, Sebastian Herold, and Jim Buckley. "Architecture consistency: State of the practice, challenges and requirements". In: *Empirical Software Engineering* 23 (2018), pp. 224–258.

[2] Domenico Amalfitano, Marco De Luca, Tiziano Santilli, Patrizio Pelliccione, and Anna Rita Fasolino. "Automated Software Architecture Design Recovery from Source Code Using LLMs". In: *European Conference on Software Architecture*. Springer. 2025, pp. 73–89.

[3] Periklis Andritsos, Panayiotis Tsaparas, Renée J Miller, and Kenneth C Sevcik. "LIMBO: Scalable clustering of categorical data". In: *International Conference on Extending Database Technology*. Springer. 2004, pp. 123–146.

[4] Nicolas Anquetil and Timothy C Lethbridge. "Recovering software architecture from the names of source files". In: *Journal of Software Maintenance: Research and Practice* 11.3 (1999), pp. 201–221. DOI: 10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1.

[5] Roberto Almeida Bittencourt, Gustavo Jansen de Souza Santos, Dalton Dario Serey Guerrero, and Gail C. Murphy. "Improving Automated Mapping in Reflexion Models Using Information Retrieval Techniques". In: *17th Working Conference on Reverse Engineering*. 2010, pp. 163–172.

[6] Joao Brunet, Roberto Almeida Bittencourt, Dalton Serey, and Jorge Figueiredo. "On the evolutionary nature of architectural violations". In: *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012, pp. 257–266.

[7] Max Brunsfeld and contributors. *Tree-sitter: An Incremental Parsing System for Programming Tools*. https://tree-sitter.github.io/tree-sitter/. Accessed: 2026-04-22. 2018.

[8] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. "NEGAR: network embedding guided architecture recovery for software systems". In: *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE. IEEE, 2022, pp. 367–376. DOI: 10.1109/apsec57359.2022.00049.

[9] Andreas Christl, Rainer Koschke, and Margaret-Anne Storey. "Automated clustering to support the reflexion method". In: *Information and Software Technology* 49.3 (2007), pp. 255–274.

[10] Paul Clements, David Garlan, Reed Little, Robert Nord, and Judith Stafford. "Documenting software architectures: views and beyond". In: *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE. 2003, pp. 740–741.

[11] Norman Cliff. "Dominance statistics: Ordinal analyses to answer ordinal questions." In: *Psychological bulletin* 114.3 (1993), p. 494.

[12] Anna Corazza, Sergio Di Martino, Valerio Maggio, and Giuseppe Scanniello. "Investigating the use of lexical information for software system clustering". In: *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE. 2011, pp. 35–44. DOI: 10.1109/csmr.2011.8.

[13] Lakshitha De Silva and Dharini Balasubramaniam. "Controlling software architecture erosion: A survey". In: *Journal of Systems and Software* 85.1 (2012), pp. 132–151. DOI: 10.1016/j.jss.2011.07.036.

[14] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. "metapath2vec: Scalable representation learning for heterogeneous networks". In: *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2017, pp. 135–144.

[15] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. "Benchmarking graph neural networks". In: *Journal of Machine Learning Research* 24.43 (2023), pp. 1–48.

[16] Ramtin Ehsani, Ahmed Aljohani, Hyunsook Do, and Yuanfang Cai. "Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development". In: *ACM Transactions on Software Engineering and Methodology* (2025). DOI: 10.1145/3820165.

[17] Michael P Fay and Michael A Proschan. "Wilcoxon-Mann-Whitney or t-test? On assumptions for hypothesis tests and multiple interpretations of decision rules". In: *Statistics Surveys* 4 (2010), p. 1.

[18] Alexander Florean, Laoa Jalal, Zipani Tom Sinkala, and Sebastian Herold. "A Comparison of Machine Learning-Based Text Classifiers for Mapping Source Code to Architectural Modules". In: *15th European Conference on Software Architecture*. 2021.

[19] Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. "A comparative analysis of software architecture recovery techniques". In: *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE. 2013, pp. 486–496. DOI: 10.1109/ase.2013.6693106.

[20] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. "Obtaining ground-truth software architectures". In: *35th International Conference on Software Engineering (ICSE)*. IEEE. 2013, pp. 901–910. DOI: 10.1109/icse.2013.6606639.

[21] Joshua Garcia, Duc Le, Daniel Link, AS Pooyan Behnamghader, Eder Figueroa Ortiz, and Nenad Medvidovic. "An empirical study of architectural change and decay in open-source software systems". In: *USC-CSSE, Tech. Rep* (2014).

[22] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. "Enhancing architectural recovery using concerns". In: *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE. 2011, pp. 552–555. DOI: 10.1109/ase.2011.6100123.

[23] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. "Diffusion improves graph learning". In: *Advances in neural information processing systems* 32 (2019).

[24] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. "Neural message passing for quantum chemistry". In: *International conference on machine learning*. PMLR. 2017, pp. 1263–1272.

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[26] Aditya Grover and Jure Leskovec. "node2vec: Scalable feature learning for networks". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 2016, pp. 855–864.

[27] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. "UniXcoder: Unified Cross-Modal Pre-training for Code Representation". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2022, pp. 7212–7225. DOI: 10.18653/v1/2022.acl-long.499.

[28] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. "Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects". In: *Proceedings of the 23rd international conference on mining software repositories*. 2026, pp. 181–193.

[29] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. "Heterogeneous graph transformer". In: *Proceedings of the web conference 2020*. 2020, pp. 2704–2710.

[30] Lawrence Hubert and Phipps Arabie. "Comparing partitions". In: *Journal of classification* 2.1 (1985), pp. 193–218. DOI: 10.1007/bf01908075.

[31] Rakhshanda Jabeen, Morgan Ericsson, Yuyao Duan, and Anna Wingkvist. *What Carries Architectural Signal? A Code-to-Architecture Mapping Study*. 2026. arXiv: 2610.41628 [cs.SE]. URL: https://arxiv.org/abs/2610.41628.

[32] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery". In: *Proceedings of the ACM on Software Engineering* 3.FSE (July 2026). DOI: 10.1145/3808187.

[33] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph convolutional networks for mapping source code entities to architectural modules". In: *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE. 2025, pp. 472–481.

[34] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Embeddings for Software Architecture Recovery". In: *Nordic Machine Intelligence* (2026). DOI: 10.5617/nmi.13153. URL: https://journals.uio.no/NMI/article/view/13153.

[35] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Representation Learning for Software Architecture Recovery". In: *ESANN*. 2026.

[36] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. *When Does the Dependency Graph Help Code-to-Architecture Mapping?* 2026. arXiv: 2610.19769 [cs.SE]. URL: https://arxiv.org/abs/2610.19769.

[37] Nafiseh Sadat Jalali, Habib Izadkhah, and Shahriar Lotfi. "Multi-objective search-based software modularization: Structural and non-structural features." In: *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 23.21 (2019). DOI: 10.1007/s00500-018-3666-z.

[38] Nils Johansson and Mauro Caporuscio. "Mapping source code to software architecture using embedding models: Insights from an analysis of data properties". In: *Information and Software Technology* 198 (2026), p. 108200. ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2026.108200. URL: https://www.sciencedirect.com/science/article/pii/S0950584926001898.

[39] Nils Johansson, Mauro Caporuscio, and Tobias Olsson. "Mapping Source Code to Software Architecture by Leveraging Large Language Models". In: *European Conference on Software Architecture*. Springer. 2024, pp. 133–149.

[40] Masoud Kargar, Ayaz Isazadeh, and Habib Izadkhah. "Semantic-based software clustering using hill climbing". In: *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE. 2017, pp. 55–60. DOI: 10.1109/csicsse.2017.8320117.

[41] Maurice G Kendall. "A new measure of rank correlation". In: *Biometrika* 30.1-2 (1938), pp. 81–93.

[42] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[43] Thomas N Kipf and Max Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016). DOI: 10.48550/arXiv.1609.02907.

[44] Thomas N Kipf and Max Welling. "Variational graph auto-encoders". In: *arXiv preprint arXiv:1611.07308* (2016). DOI: 10.48550/arXiv.1611.07308.

[45] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. "Technical Debt: From Metaphor to Theory and Practice". In: *IEEE Software* 29.6 (2012), pp. 18–21. URL: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf.

[46] Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. "An empirical study of architectural change in open-source software systems". In: *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE. 2015, pp. 235–245. DOI: 10.1109/msr.2015.29.

[47] Dong-Hyun Lee et al. "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks". In: *Workshop on challenges in representation learning, ICML*. Vol. 3. 2. Atlanta. 2013, p. 896.

[48] Jörg Lenhard, Martin Blom, and Sebastian Herold. "Exploring the suitability of source code metrics for indicating architectural inconsistencies". In: *Software Quality Journal* 27.1 (2019), pp. 241–274.

[49] Qimai Li, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[50] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. "Measuring the impact of code dependencies on software architecture recovery techniques". In: *IEEE Transactions on Software Engineering* 44.2 (2017), pp. 159–181. DOI: 10.1109/tse.2017.2671865.

[51] Onaiza Maqbool and Haroon Atique Babri. "The weighted combined algorithm: A linkage algorithm for software clustering". In: *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings.* IEEE. 2004, pp. 15–24. DOI: 10.1109/csmr.2004.1281402.

[52] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. "Distributed representations of words and phrases and their compositionality". In: *Advances in Neural Information Processing Systems* 26 (2013).

[53] Janardan Misra, KM Annervaz, Vikrant Kaulgud, Shubhashis Sengupta, and Gary Titus. "Software clustering: Unifying syntactic and semantic features". In: *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012, pp. 113–122. DOI: 10.1109/wcre.2012.21.

[54] Brian S Mitchell and Spiros Mancoridis. "On the automatic modularization of software systems using the bunch tool". In: *IEEE Transactions on Software Engineering* 32.3 (2006), pp. 193–208. DOI: 10.1109/tse.2006.31.

[55] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, Patrice Koligheu, Slim Bechikh, Kalyanmoy Deb, and Ali Ouni. "Many-objective software remodularization using NSGA-III". In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24.3 (2015), pp. 1–45. DOI: 10.1145/2729974.

[56] Sina Mohammadi and Habib Izadkhah. "A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code". In: *Information and Software Technology* 105 (2019), pp. 252–256. DOI: 10.1016/j.infsof.2018.09.001.

[57] Gail C Murphy, David Notkin, and Kevin Sullivan. "Software reflexion models: Bridging the gap between source and high-level models". In: *3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*. 1995, pp. 18–28.

[58] Fionn Murtagh and Pierre Legendre. "Ward's hierarchical agglomerative clustering method: which algorithms implement Ward's criterion?" In: *Journal of classification* 31 (2014), pp. 274–295.

[59] S. M. Naim, Kostadin Damevski, and M. Shahriar Hossain. "Reconstructing and evolving software architectures using a coordinated clustering framework". In: *Automated Software Engineering* 24.3 (2017), pp. 543–572. DOI: 10.1007/s10515-017-0211-8.

[60] Rashid Naseem, Mustafa Mat Deris, Onaiza Maqbool, and Sara Shahzad. "Euclidean space based hierarchical clusterers combinations: an application to software clustering". In: *Cluster Computing* 22.Suppl 3 (2019), pp. 7287–7311. DOI: 10.1007/s10586-017-1408-0.

[61] Rashid Naseem, Onaiza Maqbool, and Siraj Muhammad. "Cooperative clustering for software modularization". In: *Journal of Systems and Software* 86.8 (2013), pp. 2045–2062. DOI: 10.1016/j.jss.2013.03.080.

[62] Hung Viet Nguyen. *Depends: Multilingual Dependency Analysis Tool*. https://github.com/multilang-depends/depends. Accessed: 2025-08-17. 2023.

[63] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. "s4rdm3x: A Tool Suite to Explore Code to Architecture Mapping Techniques". In: *Journal of Open Source Software* 6.58 (2021).

[64] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. "To automatically map source code entities to architectural modules with Naive Bayes". In: *Journal of Systems and Software* 183 (2022), p. 111095. DOI: 10.1016/j.jss.2021.111095.

[65] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. "Representation Learning with Contrastive Predictive Coding". In: *arXiv preprint arXiv:1807.03748* (2018).

[66] Rusheng Pan, Bingcheng Mao, Tianyi Ma, and Zhenhua Ling. "ArchAgent: Scalable Legacy Software Architecture Recovery with LLMs". In: *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2026, pp. 19357–19361. DOI: 10.1109/ICASSP55912.2026.11462552. URL: https://arxiv.org/abs/2601.13007.

[67] Marios Papachristou. "Software clusterings with vector semantics and the call graph". In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019, pp. 1184–1186. DOI: 10.1145/3338906.3342483.

[68] David Lorge Parnas. "On the criteria to be used in decomposing systems into modules". In: *Communications of the ACM* 15.12 (1972), pp. 1053–1058.

[69] Saeed Parsa and Omid Bushehrian. "A new encoding scheme and a framework to investigate genetic clustering algorithms". In: *Journal of Research and Practice in Information Technology* 37.1 (2005), pp. 127–143.

[70] Babak Pourasghar, Habib Izadkhah, Ayaz Isazadeh, and Shahriar Lotfi. "A graph-based clustering algorithm for software systems modularization". In: *Information and Software Technology* 133 (2021), p. 106469. DOI: 10.1016/j.infsof.2020.106469.

[71] K. Praditwong, Mark Harman, and Xin Yao. "Software module clustering as a multi-objective search problem". In: *IEEE Transactions on Software Engineering* 37.2 (2011), pp. 264–282. DOI: 10.1109/tse.2010.26.

[72] Shiva Prasad Reddy Puchala, Jitender Kumar Chhabra, and Amit Rathee. "Ensemble clustering based approach for software architecture recovery". In: *International Journal of Information Technology* 14.4 (2022), pp. 2013–2019. DOI: 10.1007/s41870-021-00846-0.

[73] Amit Rathee and Jitender Kumar Chhabra. "Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering". In: *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer. 2017, pp. 94–106.

[74] Michele Risi, Giuseppe Scanniello, and Genoveffa Tortora. "Using fold-in and fold-out in the architecture recovery of software systems". In: *Formal Aspects of Computing* 24.3 (2012), pp. 307–330. DOI: 10.1007/s00165-011-0199-y.

[75] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen'sd for evaluating group differences on the NSSE and other surveys". In: *annual meeting of the Florida Association of Institutional Research*. Vol. 177. 34. 2006.

[76] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. "Deductive software architecture recovery via chain-of-thought prompting". In: *44th International Conference on Software Engineering: New Ideas and Emerging Results*. 2024, pp. 92–96. DOI: 10.1145/3639476.3639776.

[77] Zipani Tom Sinkala and Sebastian Herold. "InMap: Automated Interactive Code-to-Architecture Mapping Recommendations". In: *18th International Conference on Software Architecture*. 2021, pp. 173–183.

[78] Ioanna Stavropoulou, Marios Grigoriou, and Kostas Kontogiannis. "Case study on which relations to use for clustering-based software architecture recovery". In: *Empirical Software Engineering* 22.4 (2017), pp. 1717–1762. DOI: 10.1007/s10664-016-9459-z.

[79] Navid Teymourian, Habib Izadkhah, and Ayaz Isazadeh. "A fast clustering algorithm for modularization of large-scale software systems". In: *IEEE Transactions on Software Engineering* 48.4 (2020), pp. 1451–1462. DOI: 10.1109/tse.2020.3022212.

[80] Vassilios Tzerpos and Richard C Holt. "ACDC: an algorithm for comprehension-driven clustering". In: *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE. 2000, pp. 258–267. DOI: 10.1109/wcre.2000.891477.

[81] Vassilios Tzerpos and Richard C Holt. "The orphan adoption problem in architecture maintenance". In: *Working Conference on Reverse Engineering*. IEEE. 1997, pp. 76–82.

[82] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. "Graph attention networks". In: *arXiv preprint arXiv:1710.10903* (2017). DOI: 10.48550/arXiv.1710.10903.

[83] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. "Deep graph infomax". In: *arXiv preprint arXiv:1809.10341* (2018).

[84] Zhihua Wen and Vassilios Tzerpos. "An effectiveness measure for software clustering algorithms". In: *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE. 2004, pp. 194–203. DOI: 10.1109/wpc.2004.1311061.

[85] Frank Wilcoxon. "Individual comparisons by ranking methods". In: *Biometrics bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.

[86] Kaiyuan Yang, Junfeng Wang, Zhiyang Fang, Peng Wu, and Zihua Song. "Enhancing software modularization via semantic outliers filtration and label propagation". In: *Information and Software Technology* 145 (2022), p. 106818. DOI: 10.1016/j.infsof.2021.106818.

[87] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. "Software architecture recovery with information fusion". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023, pp. 1535–1547. DOI: 10.1145/3611643.3616285.

[88] Wenting Zhao, Wuxia Jin, Yiran Zhang, Ming Fan, Haijun Wang, Li Li, Yang Liu, and Ting Liu. "Software Architecture Recovery Augmented with Semantics". In: *IEEE Transactions on Software Engineering* (2025).

[89] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. "Beyond homophily in graph neural networks: Current limitations and effective designs". In: *Advances in neural information processing systems* 33 (2020), pp. 7793–7804.